

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในโครงการ

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการวิจัย ไม่ว่าจะเป็นพื้นฐานของแนวทางการออกแบบเว็บแอปพลิเคชัน เทคโนโลยีเว็บเซอร์วิส โครงสร้างคอตเน็ตเทคโนโลยี ทฤษฎีการค้นหาเส้นทางต่างๆ รวมทั้งเครื่องมือต่างๆที่ใช้ในการศึกษานี้ ซึ่งเนื้อหาทั้งหมดนี้จำเป็นสำหรับการศึกษา และประเมินประสิทธิภาพของการพัฒนาเว็บแอปพลิเคชันที่ทำการทดลองนี้

2.1. แนวทางการพัฒนาเว็บแอปพลิเคชัน

ในปัจจุบันเมื่ออินเทอร์เน็ตมีการใช้งานอย่างกว้างขวางขึ้น ความต้องการประยุกต์ใช้สิ่งใหม่ ๆ บนอินเทอร์เน็ตจึงได้รับการพัฒนามากขึ้น รวมทั้งเว็บไซต์ ก็เป็นอีกหนึ่งเทคโนโลยีที่มีการพัฒนาอย่างต่อเนื่องเพราะฉะนั้นการให้บริการเว็บในปัจจุบันไม่เพียงแต่ความหลากหลายของข้อมูล และรูปแบบที่ให้บริการเท่านั้น ตัวเว็บไซต์เองก็มีขนาดใหญ่และมีความซับซ้อนมากขึ้นอีกด้วย

2.1.1. รูปแบบการพัฒนาเว็บไซต์

สำหรับการพัฒนาเว็บนั้นเราสามารถแบ่งระดับออกได้เป็นสามระดับคือ

- การพัฒนาเว็บสำหรับให้บริการข้อมูล
- การพัฒนาเว็บแบบโต้ตอบได้
- การพัฒนาเว็บแอปพลิเคชัน

2.1.1.1. การพัฒนาเว็บสำหรับให้บริการข้อมูล

เป็นการสร้างเว็บไซต์ที่มีเป้าหมายหลักสำหรับการให้บริการข้อมูล ซึ่งเป็นการให้บริการแบบทางเดียว โดยผู้พัฒนาสามารถสร้างเว็บไซต์ขึ้นมาได้ในทันทีโดยไม่ต้องผ่านการกระบวนการออกแบบ แต่ถ้าเว็บมีขนาดที่ใหญ่มากก็อาจจะจำเป็นต้องมีการออกแบบโครงสร้างเว็บไซต์ โดยในการออกแบบเว็บไซต์ประเภทนี้ผู้ออกแบบจะต้องเน้นไปที่ความสะดวกของผู้ใช้ในการเข้าถึงข้อมูล ซึ่งจำเป็นต้องออกแบบโครงสร้างของเว็บให้สามารถเข้าถึงแต่ละส่วนได้ง่ายและไม่ซับซ้อนจนเกินไป

2.1.1.2. การพัฒนาเว็บแบบโต้ตอบได้

เป็นการพัฒนาเว็บไซต์ที่มีความสามารถในการรับข้อมูลจากผู้ใช้งานมาประมวลผล หรือนำมาเก็บไว้เพื่อนำมาใช้ในอนาคต พร้อมทั้งมีการแสดงหน้า เอชทีเอ็มแอลแบบไดนามิกตามข้อมูลที่ผู้ใช้ป้อนเข้ามา โดยทั่วไปแล้วการพัฒนาเว็บแบบนี้จะต้องมีฟังก์ชันการให้บริการข้อมูล

อยู่ด้วยเป็นหลัก แต่จะมีการเพิ่มความสามารถในการรับข้อมูลจากผู้ใช้ เช่น บอร์ดแสดงความคิดเห็น (Web Board) หรือการฝากข้อความ ส่วนใหญ่แล้วการพัฒนาเว็บแบบนี้จะมีการประมวลผลและการเก็บข้อมูลที่ไม่ซับซ้อนมาก โปรแกรมที่สร้างนั้นก็จะมีขนาดไม่ใหญ่และไม่จำเป็นต้องใช้ระบบฐานข้อมูล การออกแบบเว็บประเภทนี้นั้น ผู้ออกแบบจะต้องคำนึงถึงการออกแบบคล้ายกับการพัฒนาเว็บสำหรับให้บริการข้อมูลรวมทั้งยังต้องมีการทำโมเดลแสดงอัลกอริทึมเพื่อแสดงการทำงานของโปรแกรมที่มีอยู่ในเว็บเพื่อให้สามารถกลับมาแก้ไขโปรแกรมได้ง่ายอีกด้วย

2.1.1.3. การพัฒนาเว็บแอปพลิเคชัน

เป็นการพัฒนาเว็บไซต์ที่มีความซับซ้อนมากที่สุด เมื่อเทียบกับการพัฒนาจากทั้งสองแบบที่กล่าวมา โดยเว็บแอปพลิเคชันนั้นจะประกอบไปด้วยส่วนสำหรับให้บริการข้อมูล ส่วนที่โต้ตอบกับผู้ใช้ และส่วนที่ใช้ในการทำการประมวลผล โดยทางผู้พัฒนานั้นอาจจะต้องใช้ภาษาในการเขียนโปรแกรมเฉพาะเพื่อให้สามารถควบคุมการประมวลผลได้ง่าย นอกจากนี้แล้วผู้พัฒนายังต้องคำนึงการติดต่อสื่อสารกับภายนอก และส่วนการเก็บข้อมูลที่มีความซับซ้อน ซึ่งบางครั้งอาจจะจำเป็นต้องมีการใช้ ระบบการจัดการฐานข้อมูลมาช่วยด้วย สำหรับการพัฒนาเว็บแอปพลิเคชันผู้พัฒนาจะต้องทำการโมเดลที่แสดงการทำงานของเว็บแอปพลิเคชันเพิ่มขึ้นมา โดยต้องมีการออกแบบโครงสร้างส่วนประกอบ และการออกแบบฐานข้อมูลด้วย

2.1.2. สถาปัตยกรรมในการติดตั้งเว็บไซต์

เราสามารถที่จะทำการแบ่งสถาปัตยกรรมในการติดตั้งเว็บไซต์ออกได้เป็น 3 แบบ คือ

2.1.2.1. สถาปัตยกรรมแบบซิงเกิลเทียร์(Single-Tier)

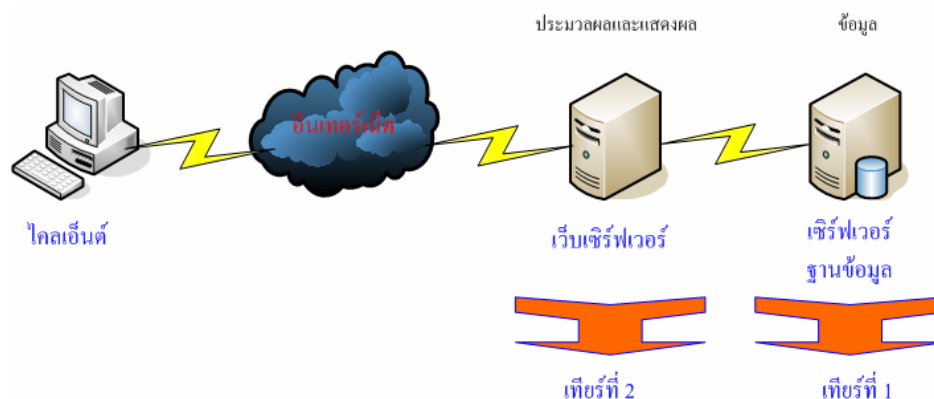
เป็นสถาปัตยกรรมที่ประกอบไปด้วยเว็บเซิร์ฟเวอร์ และไคลเอ็นต์ โดยเว็บเซิร์ฟเวอร์นั้นจะทำหน้าที่ในการให้บริการข้อมูลของเว็บรวมทั้งการประมวลผลต่างๆ ส่วนไคลเอ็นต์นั้นจะแสดงผลต่อผู้ใช้และอาจจะทำการส่งข้อมูลจากผู้ใช้กลับไปให้เซิร์ฟเวอร์เพื่อประมวลผลและเก็บไว้ด้วย ดังรูปที่ 2.1.



รูปที่ 2.1. สถาปัตยกรรมของเว็บแบบซิงเกิลเทียร์(Single-Tier)

2.1.2.2. สถาปัตยกรรมแบบทูเทียร์(Two-Tier)

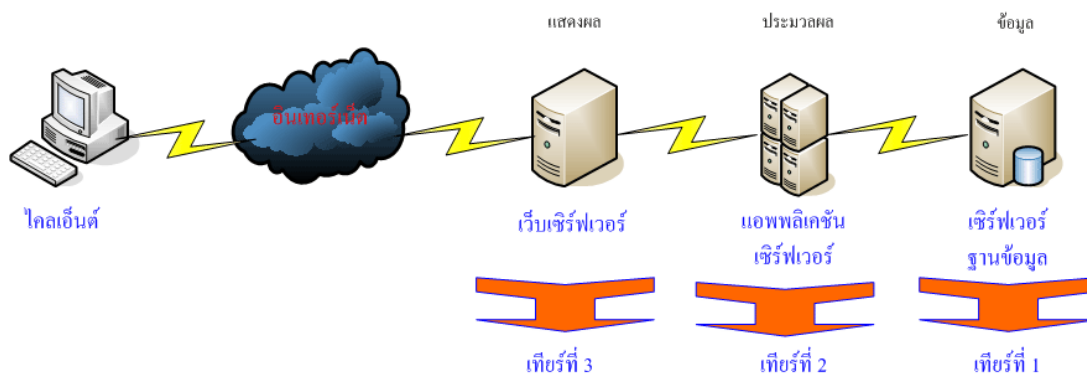
เป็นสถาปัตยกรรมที่ประกอบไปด้วยเว็บเซิร์ฟเวอร์ ไคลเอ็นต์รวมทั้งมีการเพิ่มในส่วน
ของระบบฐานข้อมูล ซึ่งเหมาะสำหรับเว็บแอปพลิเคชันที่เน้นการเก็บข้อมูลของผู้ใช้เป็นหลัก ดัง
รูปที่ 2.2.



รูปที่ 2.2. สถาปัตยกรรมของเว็บแบบทูเทียร์(Two-Tier)

2.1.2.3. สถาปัตยกรรมแบบทรีเทียร์(Three-Tier)

เป็นสถาปัตยกรรมที่มีการแยกระบบออกเป็น ระบบฐานข้อมูล แอปพลิเคชันเซิร์ฟเวอร์
เว็บเซิร์ฟเวอร์และไคลเอ็นต์ โดยในส่วนของแอปพลิเคชันเซิร์ฟเวอร์จะทำหน้าที่ในการ
ประมวลผลหรือทำธุรกรรมต่างๆ เป็นหลัก ส่วนเว็บเซิร์ฟเวอร์นั้นจะนำผลลัพธ์ที่ได้จากการ
ประมวลผลจากแอปพลิเคชันเซิร์ฟเวอร์มาจัดหน้าตา เพื่อส่งไปให้ยังไคลเอ็นต์แสดงผลต่อไป ดัง
รูปที่ 2.3. เนื่องจากสถาปัตยกรรมทรีเทียร์จะมีการแบ่งส่วนประมวลผลและส่วนของการแสดงผล
แยกออกจากกันอย่างชัดเจน จึงทำให้ผู้พัฒนาสามารถที่จะทำการเปลี่ยนแปลงและแก้ไขส่วน
ประมวลผลได้โดยไม่กระทบต่อส่วนแสดงผล หรือมีการเปลี่ยนแปลงหน้าตาส่วนแสดงผลก็จะ
ไม่มีผลกระทบต่อส่วนประมวลผลเป็นต้น นอกจากนี้สถาปัตยกรรมแบบทรีเทียร์นี้ยังสนับสนุน
ความสามารถในการขยายระบบ และความสามารถในการแบ่งภาระงานอีกด้วย



รูปที่ 2.3. สถาปัตยกรรมของเว็บแบบทรีเทียร์(Three-Tier)

2.1.3. ภาษายูเอ็มแอลกับเว็บ

ภาษายูเอ็มแอล (Unified Modeling Language) ได้รับการพัฒนาขึ้นโดยบริษัท เรชันแนล ซอฟต์แวร์ (Rational Software) ซึ่งเป็นภาษาที่ใช้ในการอธิบายซอฟต์แวร์ โดยจะมีการนำภาษา ยูเอ็มแอลนี้มาใช้ตั้งแต่ เริ่มพัฒนาซอฟต์แวร์ คือ การเก็บความต้องการ การวิเคราะห์ การ ออกแบบ พัฒนา จนถึงขั้นตอนของการทดสอบ โดยส่วนประกอบหลักของภาษายูเอ็มแอลคือ แผนภาพ (Diagram) และ สัญลักษณ์ (Notation) ที่มีความหมาย

ถึงแม้ว่าการประยุกต์ใช้ภาษายูเอ็มแอลกับการพัฒนาเว็บไซต์นั้นจะไม่สามารถทำได้ โดยตรง แต่ถ้าผู้พัฒนาทราบถึงวิธีทำความเข้าใจและตีความหมายของสัญลักษณ์ในภาษายูเอ็ม แอลให้ตรงกับส่วนประกอบของเว็บไซต์ก็สามารถทำได้ไม่ยาก โดยผู้พัฒนาจะต้องมีความรู้ความ เข้าใจในตัวเทคโนโลยีที่ใช้ในการพัฒนาเว็บไซต์นั้น อีกทั้งต้องทำความเข้าใจกับ แนวความคิด ความหมาย และสัญลักษณ์ของภาษายูเอ็มแอลพื้นฐาน จากนั้นจึงทำความเข้าใจว่าสัญลักษณ์และ ความหมายที่มีในภาษายูเอ็มแอล นั้นจะนำไปโมเดลส่วนประกอบของเว็บไซต์ได้อย่างไร

โดยในตัวภาษายูเอ็มแอลก็มีแผนภาพมากมายคือ

2.1.3.1. แผนภาพยูสเคส (Use Case diagram)

เป็นแผนภาพที่ใช้ในการแสดงความต้องการในแง่ของฟังก์ชันการทำงาน (Functional requirements) ที่สำคัญของระบบ และอาจจะรวมถึงความต้องการที่ไม่เป็นฟังก์ชัน (Nonfunctional requirements) ที่สำคัญด้วย ส่วนแผนภาพยูสเคสในมุมมองของผู้ใช้นั้นจะเป็นแผนภาพที่แสดง ฟังก์ชันการทำงานหลักของซอฟต์แวร์ ทีมพัฒนาสามารถใช้แผนภาพยูสเคสนี้เป็นเอกสารหลักใน การสื่อสารกับผู้ใช้ซอฟต์แวร์ และยังสามารถใช้สื่อสารระหว่างสมาชิกในทีมพัฒนาได้อีกด้วย

สำหรับในแผนภาพยูสเคสจะประกอบไปด้วยส่วนที่สำคัญอยู่ 3 ส่วน คือ ยูสเคส(use case), แอคเตอร์(actor), และความสำคัญระหว่างยูสเคส กับ แอคเตอร์

2.1.3.1.1. ยูสเคส จะเป็นตัวบ่งบอกถึงฟังก์ชันที่สำคัญซึ่งระบบสามารถทำได้ โดยการเขียนข้อความภายในยูสเคสนั้น ควรที่จะสื่อภาพให้เห็นได้ว่ายูสเคสนั้นเกี่ยวข้องกับอะไรกับการทำซอฟต์แวร์ และโดยทั่วไปแล้วข้อความในยูสเคสนั้นควรที่จะเป็นคำกริยา เช่น ยูสเคสสำหรับการจองตั๋วเครื่องบิน,เปลี่ยนแปลงเที่ยวบิน เป็นต้น โดยสัญลักษณ์จะเป็นดังรูปที่ 2.4.



รูปที่ 2.4. สัญลักษณ์ที่ใช้แทน ยูสเคส

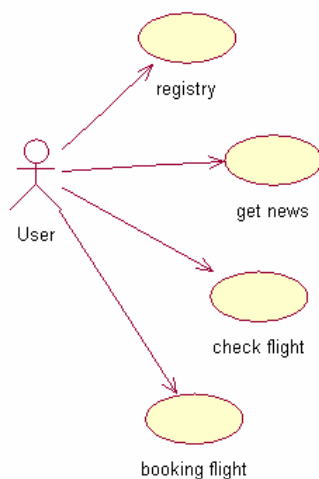
2.1.3.1.2. แอคเตอร์ เป็นตัวบ่งบอกถึงสิ่งแวดล้อมภายนอกที่เกี่ยวข้องกับซอฟต์แวร์ เช่น ลูกค้า,ผู้ใช้งานระบบ หรือ ผู้ดูแลระบบ เป็นต้น นอกจากนี้แอคเตอร์ไม่จำเป็นที่จะต้องเป็นมนุษย์เท่านั้น แต่ยังสามารถที่จะยังเป็นระบบได้ หรือเป็นอุปกรณ์ที่อยู่นอกซอฟต์แวร์ได้ โดยสัญลักษณ์จะเป็นดังรูปที่ 2.5.



รูปที่ 2.5. สัญลักษณ์ที่ใช้แทน แอคเตอร์

2.1.3.1.3. ความสัมพันธ์ระหว่างยูสเคส กับ แอคเตอร์ จะใช้เส้นตรงลากจาก แอคเตอร์มายังยูสเคสที่เกี่ยวข้องโดยความสำคัญระหว่างยูสเคสกับแอคเตอร์นั้นเป็นความสัมพันธ์แบบแมนนี่ทูแมนนี่ (many-to-many) คือแอคเตอร์หนึ่งอาจจะมีเกี่ยวข้องกับหลายยูสเคสได้ และยูสเคสหนึ่งยูสเคสก็อาจจะเกี่ยวข้องกับแอคเตอร์หลายแอคเตอร์ได้ ดังรูป 2.6.

นอกจากนี้แล้วการกำหนดความสัมพันธ์ระหว่างยูสเคสนั้นก็ยังสามารถทำได้ เพื่อให้เป็นการกำหนดรายละเอียดของยูสเคสภายในมุมมองของทีมพัฒนา โดยอาจจะมี การสืบทอดคุณสมบัติให้กับยูสเคสจากตัวหนึ่งสู่อีกตัวหนึ่งได้

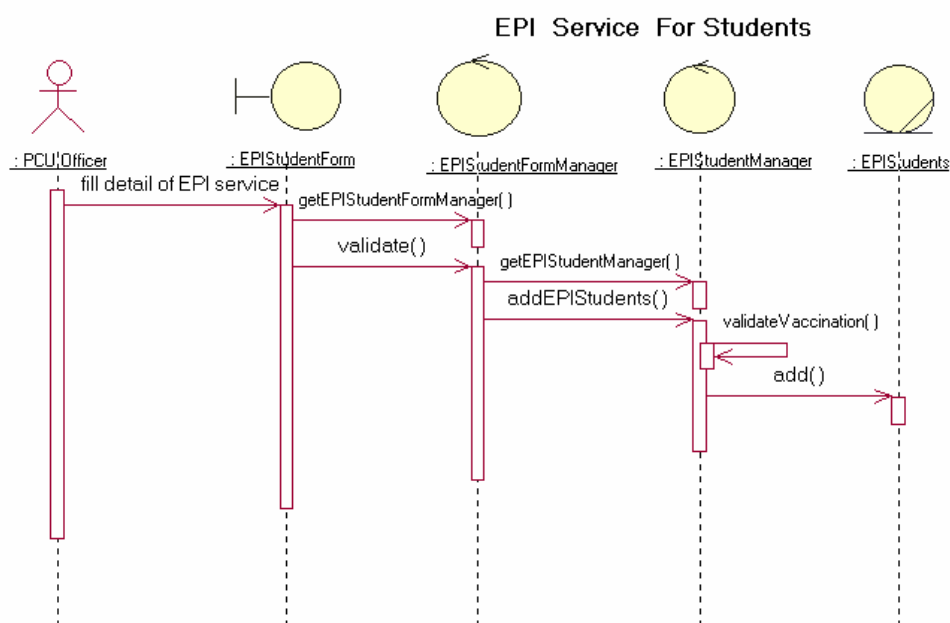


รูปที่ 2.6. ตัวอย่างแผนภาพยูสเคส

2.1.3.2. แผนภาพแสดงการทำงาน(Scenario diagram)

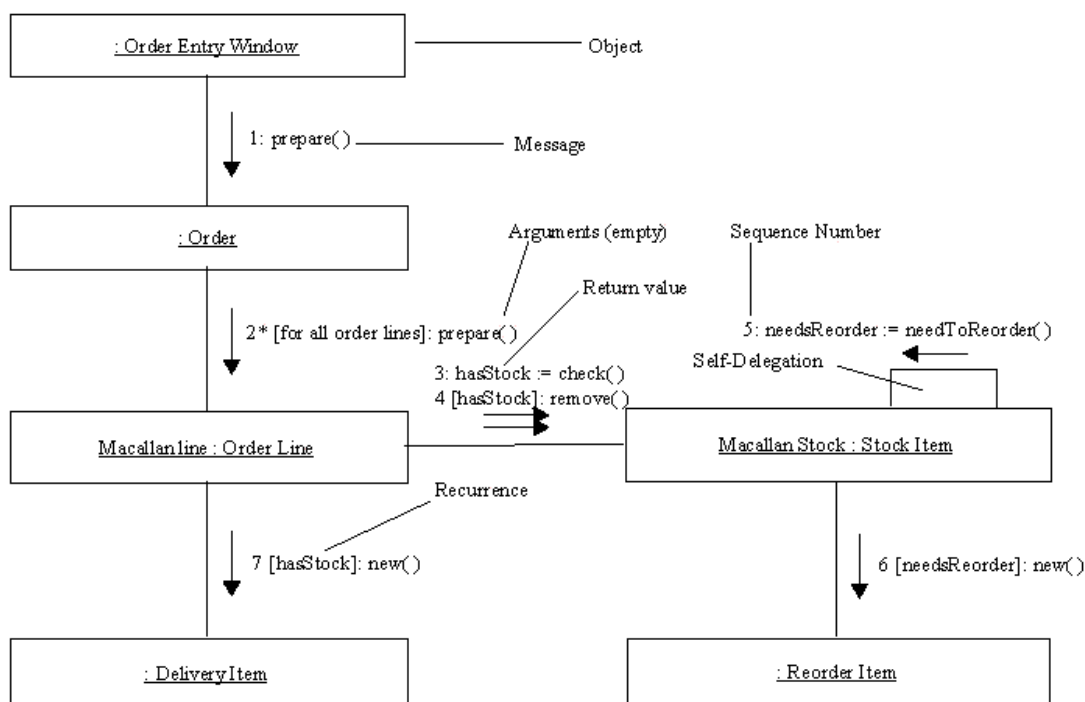
เป็นขั้นตอนของการที่จะอธิบายขั้นตอนการทำงานในกรณีหนึ่งของยูสเคส โดยจะเลือกเหตุการณ์ที่สำคัญและมีบทบาทหลักต่อยูสเคส เพื่อมาอธิบายให้ทั้งผู้ใช้และทีมพัฒนาซอฟต์แวร์เข้าใจ ซึ่งการอธิบายการทำงานของยูสเคสในกรณีหนึ่งนั้นสามารถทำได้โดยใช้แผนภาพที่ภาษา ยูเอ็มแอลมีให้ซึ่งประกอบด้วย แผนภาพแสดงลำดับการทำงาน (Sequence Diagram) และ แผนภาพการทำงานร่วมกัน (Collaboration Diagram)

2.1.3.1.1 . แผนภาพแสดงลำดับการทำงาน (Sequence Diagram) นั้นจะแสดงการลำดับขั้นตอนของการ และอาจจะมีเงื่อนไขทางด้านเวลาเข้ามาเกี่ยวข้องด้วย สำหรับส่วนประกอบภายในก็จะประกอบไปด้วยส่วน แอคเตอร์, คลาสหรือออบเจกต์ และเมสเสจ (Message) ที่ไหลอยู่ภายในซอฟต์แวร์ โดยเมสเสจที่อยู่ด้านบนนั้นจะเป็นเมสเสจที่เกิดขึ้นก่อนเมสเสจที่อยู่ด้านล่าง และเมสเสจนั้นจะลากจากแอคเตอร์ คลาสหรือออบเจกต์ที่เป็นต้นกำเนิดไปยังปลายทางเพื่อบ่งบอกผู้เรียกและผู้ให้บริการ ดังรูป 2.7.



รูปที่ 2.7. ตัวอย่างแผนภาพแสดงลำดับการทำงาน

2.1.3.1.2. แผนภาพการทำงานร่วมกัน มีวัตถุประสงค์เพื่อผู้ใช้แสดงการทำงานสำหรับยูสเคสในกรณีหนึ่งเช่นเดียวกับแผนภาพแสดงลำดับการทำงาน แต่จะแตกต่างกันที่ การนำเสนอของแผนภาพแสดงการทำงานร่วมกันนั้นจะเน้นที่การแสดงโครงสร้างในการทำงานภายในยูสเคสมากกว่าที่จะแสดงลำดับ แต่กระนั้นแล้ว ในแผนภาพแสดงการทำงานร่วมกันนี้ก็มีการแสดงลำดับของการทำงานไว้ด้วยเช่นกัน สำหรับภายในแผนภาพแสดงการทำงานร่วมกันนั้นก็มีส่วนประกอบคือ แอคเตอร์, คลาสหรือออบเจกต์, ความสัมพันธ์ระหว่างกัน และเมสเซจ (Message) โดยที่เมสเซนนั้นจะมีการบ่งบอกลำดับการเกิดขึ้นด้วยว่าเมสเซจใดเกิดขึ้นก่อนเมสเซจใด ดังรูป 2.8.



รูปที่ 2.8. ตัวอย่างแผนภาพแสดงการทำงานร่วมกัน

2.1.3.3. แผนภาพออบเจกต์ (Object diagram)

แผนภาพออบเจกต์นั้นเป็นแผนภาพที่ใช้ในการแสดงอินสแตนซ์ (Instance) ของส่วนประกอบภายในซอฟต์แวร์ โดยส่วนใหญ่แล้วการพัฒนาซอฟต์แวร์จะไม่ได้เขียนแผนภาพออบเจกต์ขึ้นมาโดยตรง แต่จะใช้แผนภาพแสดงการทำงานร่วมกันแทน ทั้งนี้เนื่องจากแผนภาพแสดงการทำงานร่วมกันนั้นจะใช้ในการนำเสนอการสื่อสารระหว่างออบเจกต์ภายในซอฟต์แวร์อยู่แล้ว ส่งผลให้เครื่องมือในการเขียนแผนภาพของยูเอ็มแอลบางตัวไม่สนับสนุนการเขียนแผนภาพออบเจกต์

สำหรับส่วนประกอบภายในแผนภาพนั้นก็จะประกอบด้วย สัญลักษณ์แทนออบเจกต์ ซึ่งจะเป็นกล่องสี่เหลี่ยมพร้อมด้วยชื่อของออบเจกต์ซึ่งอยู่ภายใน และความสัมพันธ์ระหว่างออบเจกต์ โดยความสัมพันธ์ระหว่างออบเจกต์ที่สำคัญในแผนภาพออบเจกต์นั้นประกอบด้วย

2.1.3.3.1. ความสัมพันธ์แบบมีส่วนร่วม (Association)

เป็นความสัมพันธ์ระหว่างออบเจกต์ที่เกิดขึ้นเนื่องจากออบเจกต์ 2 ออบเจกต์มีความเกี่ยวข้องกัน ดังเช่นมีการเรียกใช้บริการ หรือมีการส่งเมสเซจถึงกันเป็นต้น

สำหรับสัญลักษณ์ที่ใช้แทนความสัมพันธ์ระหว่างออบเจกต์แบบมีส่วนร่วมนั้นจะเป็นเส้นตรงที่ลากระหว่างออบเจกต์ที่สัมพันธ์กัน นอกจากนั้นยังสามารถที่จะแสดงทิศทางในการ

เรียกใช้บริการหรือส่งแมสเสจได้ด้วยวิธีการเติมหัวลูกศร โดยออบเจกต์ต้นทางจะเป็นผู้เรียกใช้บริการหรือเป็นผู้ส่งแมสเสจไปให้ ดังรูป 2.9.

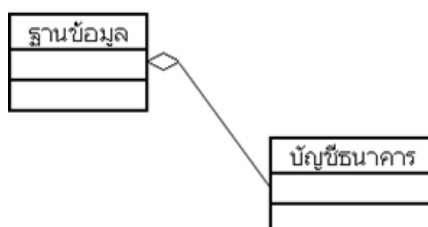


รูปที่ 2.9. ตัวอย่าง ความสัมพันธ์แบบมีส่วนร่วม (Association)

2.1.3.3.2. ความสัมพันธ์แบบเป็นส่วนหนึ่งของ (Aggregation)

เป็นความสัมพันธ์ที่ออบเจกต์หนึ่งแสดงความเป็นเจ้าของกับอีกออบเจกต์หนึ่ง นั่นคือนอกจากออบเจกต์หนึ่งที่มีความสัมพันธ์แบบเป็นส่วนหนึ่งนั้นจะมีการเรียกใช้บริการหรือมีการแลกเปลี่ยนแมสเสจกันแล้ว ออบเจกต์ที่เป็นเจ้าของจะมีหน้าที่ที่จะต้องรับผิดชอบในการสร้างออบเจกต์ที่เป็นส่วนประกอบอีกด้วย

สำหรับสัญลักษณ์ที่ใช้แทนความสัมพันธ์แบบเป็นส่วนหนึ่งนั้นจะเป็นเส้นตรงที่ลากระหว่างออบเจกต์เช่นกัน แต่ปลายเส้นที่ออบเจกต์ที่เป็นเจ้าของจะมีสัญลักษณ์ของสี่เหลี่ยมปรากฏอยู่ด้วย เพื่อใช้ในการแสดงความเป็นเจ้าของ ดังรูป 2.10.



รูปที่ 2.10. ตัวอย่าง ความสัมพันธ์แบบเป็นส่วนหนึ่ง (Aggregation)

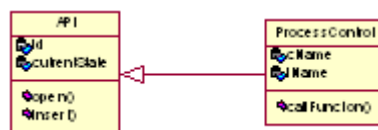
2.1.3.4 . แผนภาพคลาส (Class diagram)

เป็นแผนภาพคลาสี่จะเป็นแผนภาพหลักสำหรับกระบวนการออกแบบและพัฒนา รวมถึงการทดสอบด้วย โดยแผนภาพคลาสนั้นประกอบไปด้วยคลาส และความสัมพันธ์ระหว่างคลาส สำหรับสัญลักษณ์ที่ใช้แทนคลาสนั้นจะเป็นสี่เหลี่ยม ซึ่งจะแบ่งออกเป็นสามส่วนคือ ชื่อของคลาส เมธอด (Method) ซึ่งเป็นส่วนที่จะบ่งบอกถึงพฤติกรรมของคลาส และแอททริบิวต์ (Attribute) ซึ่งจะบ่งบอกถึงข้อมูลที่เก็บอยู่ในคลาส และความสัมพันธ์ของคลาสนั้นก็จะคล้ายกับความสัมพันธ์ของออบเจกต์ โดยความสัมพันธ์ระหว่างคลาสนั้นที่มีความสัมพันธ์คือ

- ความสัมพันธ์แบบมีส่วนร่วม (Association)

- ความสัมพันธ์แบบเป็นส่วนหนึ่งของ (Aggregation)
- ความสัมพันธ์แบบสืบทอดคุณสมบัติ (Generalization หรือ Inheritance)

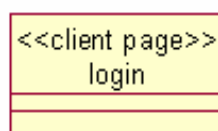
โดยความสัมพันธ์แบบสืบทอดคุณสมบัติเป็นความสัมพันธ์ที่เกิดขึ้นเมื่อมีคลาสหนึ่งสืบทอด โดยเราจะเรียกคลาสที่ให้การสืบทอดคุณสมบัติว่า ซุปเปอร์คลาส (Super class) และเรียกคลาสที่ทำการสืบทอดคุณสมบัติมาว่า ซับคลาส (Sub class) เนื่องจากซับคลาสนั้นจะสืบทอดคุณสมบัติจาก ซุปเปอร์คลาส และจะส่งผลให้ซับคลาสนั้นมีคุณสมบัติทุกอย่างเหมือนกับซุปเปอร์คลาส และยังสามารถที่จะทำการเพิ่มคุณสมบัติเข้าไปได้ตามที่ต้องการอีกด้วย ดังรูป 2.11.



รูปที่ 2.11. ตัวอย่างการกำหนดความสัมพันธ์แบบสืบทอดคุณสมบัติเพื่อเป็นการเจาะจงซับคลาสให้มีความเฉพาะมากขึ้น

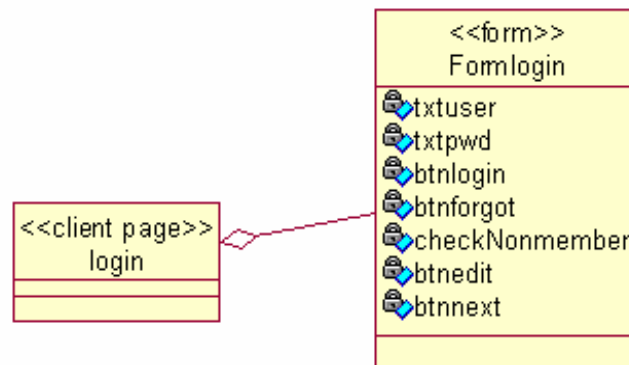
แต่ในการออกแบบแผนภาพคลาสของเว็บแอปพลิเคชันนั้น จะมีคลาสต่างๆที่เกี่ยวข้องกับเว็บแอปพลิเคชัน คือ

- **ไคลเอนต์เพจ (client page)** เป็นคลาสที่ใช้แทนเพจฝั่งไคลเอนต์ซึ่งโดยทั่วไปจะสืบทอดมาจากบราวเซอร์คลาสในขั้นตอนของการวิเคราะห์ โดยไคลเอนต์เพจนี้สามารถที่จะลิงก์ไปยังไคลเอนต์เพจอื่นๆ หรือเว็บเซิร์ฟเวอร์อื่นๆ ได้ สามารถมีฟอร์ม หรือสคริปและออบเจกต์ที่ฝั่งไคลเอนต์ ดังรูป 2.12.



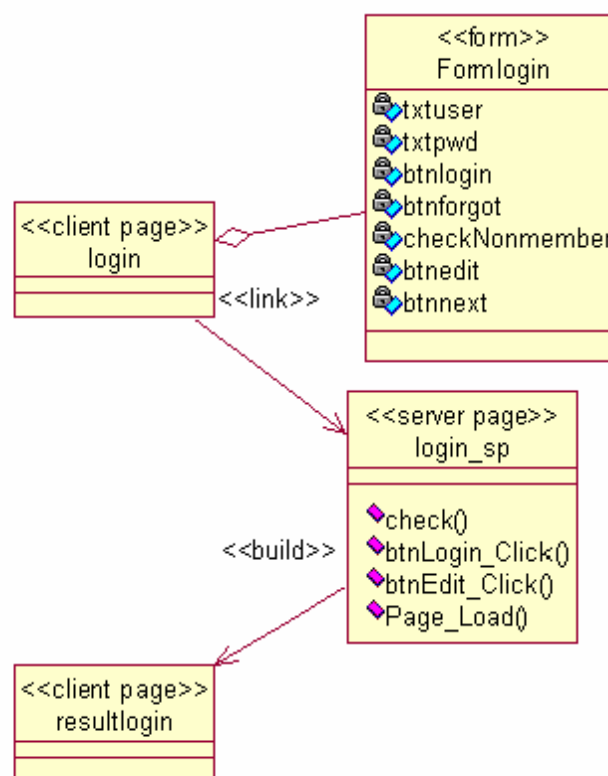
รูปที่ 2.12. ตัวอย่างไคลเอนต์เพจ

- **ฟอร์ม(Form)** โดยทั่วไปคลาสฟอร์มจะประกอบเป็นส่วนหนึ่งของไคลเอนต์เพจ โดยคลาสฟอร์มนั้นจะใช้เพื่อรับข้อมูลของผู้ใช้ ซึ่งภายในฟอร์มจะมีออบเจกต์สำหรับรับข้อมูลเช่นฟิลด์และปุ่มซึ่งใช้ในการส่งข้อมูลและเรียกให้เซิร์ฟเวอร์เพจทำงานได้ ดังรูป 2.13.



รูปที่2.13. ตัวอย่างฟอร์ม

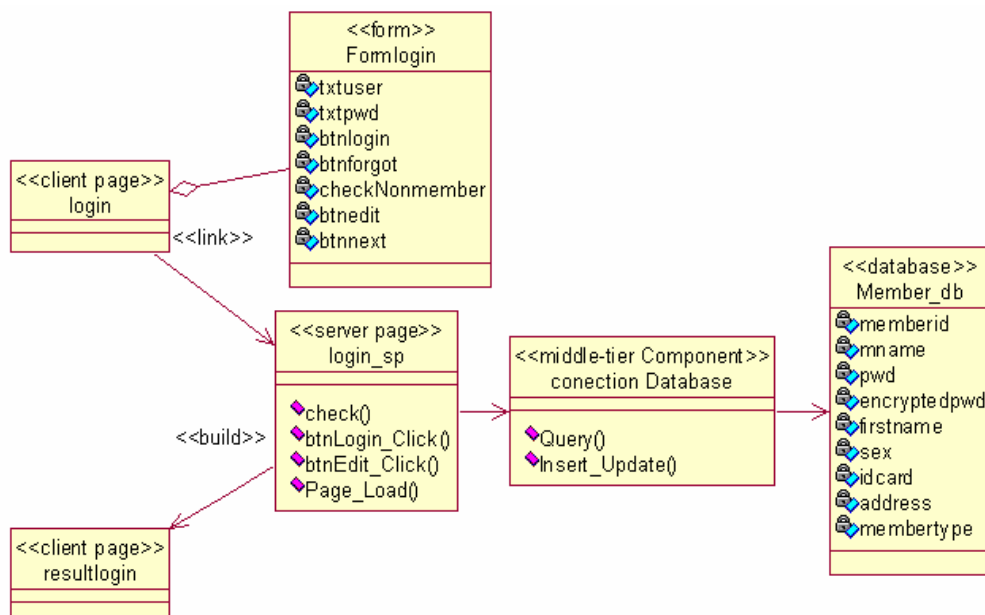
- เซิร์ฟเวอร์เพจ (server page) ซึ่งในการกำหนดเซิร์ฟเวอร์เพจนั้น เราสามารถที่จะกำหนดเซิร์ฟเวอร์เพจเป็นคลาสๆหนึ่งในเว็บแอปพลิเคชันได้ โดยคลาสเซิร์ฟเวอร์เพจนี้จะมีความสัมพันธ์กับไคลเอนต์ในกรณีที่เราใช้งาน และอาจจะมีการไปสร้างคลาสที่เป็นคอมโพเนนต์ที่เกี่ยวกับกลาง ดังรูป 2.14.



รูปที่2.14. ตัวอย่างเซิร์ฟเวอร์เพจ

- คลาสคอมโพเนนต์ซึ่งทำงานอยู่ที่เทียร์กลาง (Middle-Tier Component) เป็นคลาสที่ใช้ในการสร้างออบเจกต์เมื่อเซิร์ฟเวอร์เพจถูกเรียกใช้งานเพื่อประมวลผล หรือเมื่อต้องการติดต่อกับฐานข้อมูล ซึ่งการนำเอาคลาสคอมโพเนนต์ซึ่งทำงานอยู่ที่เทียร์กลาง (Middle-Tier-Component) นำไปใช้ในการออกแบบเว็บแอปพลิเคชันนี้จะช่วยให้ผู้พัฒนาสามารถที่จะทำการเปลี่ยนแปลงคลาสคอมโพเนนต์ที่ทำงานที่เทียร์กลางนี้ได้โดยจะมีผลกระทบกับเซิร์ฟเวอร์เพจ

นั้นน้อยมาก ซึ่งจะเหมาะกับการทำเว็บแอปพลิเคชันที่คำนึงถึงหลักการของการบำรุงรักษา (Maintenance) ดังรูป 2.15.

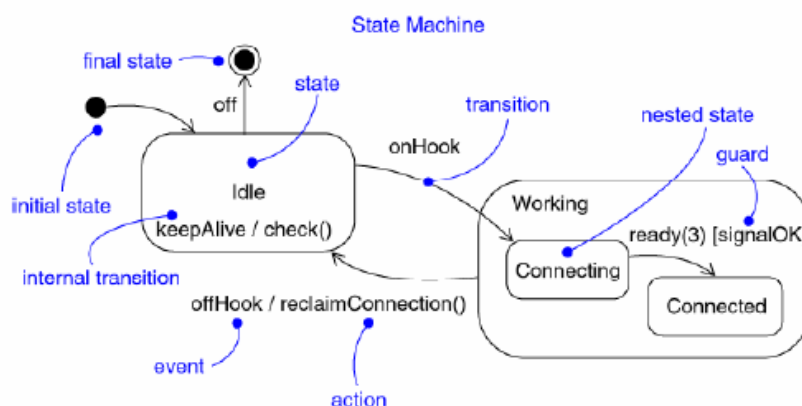


รูปที่2.15 ตัวอย่างคลาสคอมโพเนนท์ซึ่งทำงานอยู่ที่เซิร์ฟเวอร์กลาง

2.1.3.5. แผนภาพสถานะการทำงาน (State Diagram)

เป็นแผนภาพที่ใช้ในการอธิบายพฤติกรรมของซอฟต์แวร์ ส่วนประกอบภายใน หรือแม้แต่คลาสในแผนภาพของภาษายูเอ็มแอล โดยแผนภาพสถานะนี้จะเน้นที่การนำเสนอการทำงานภายใต้สถานะการทำงานของซอฟต์แวร์ทั้งหมดในทุกกรณีที่มี

สำหรับส่วนประกอบในแผนภาพสถานะนั้นประกอบไปด้วย จุดเริ่มต้น จุดจบ และสถานะ เส้นทาง การเปลี่ยนแปลงสถานะตามเหตุการณ์ที่มีเข้ามาและการกำหนดเงื่อนไขสำหรับการเปลี่ยนสถานะ ดังรูป 2.16.

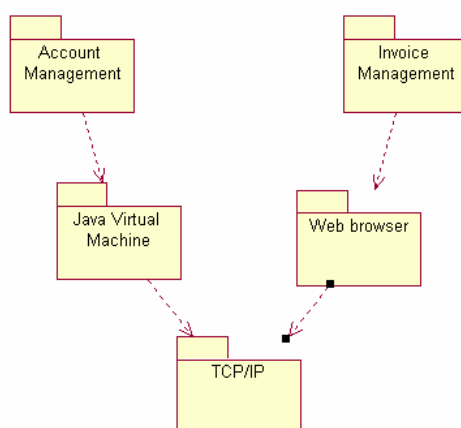


รูปที่2.16. ตัวอย่างแผนภาพสถานะ

2.1.3.5. แผนภาพคอมโพเนนต์ (Component diagram)

เป็นแผนภาพที่ใช้ในการอธิบายส่วนประกอบที่จะมีในระบบโดยเฉพาะส่วนของซอฟต์แวร์ สำหรับคอมโพเนนต์ในระบบนั้นจะหมายถึงระบบย่อยหรือแพ็คเกจ เราสามารถกล่าวได้ว่าคอมโพเนนต์นั้นเกิดขึ้นจากการรวมคลาสหลายๆคลาสที่มีหน้าที่หรือความรับผิดชอบคล้ายกันมาอยู่รวมกันในคอมโพเนนต์เดียวกันเพื่อให้ง่ายต่อการจัดการและดูแล

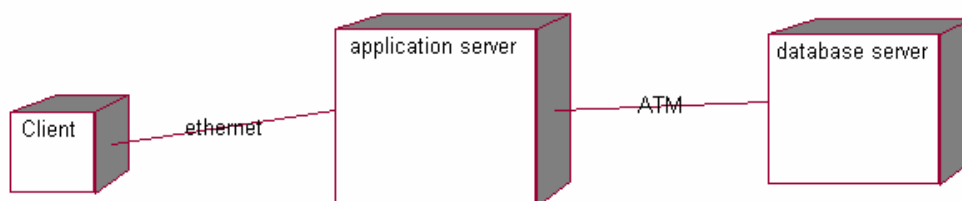
แผนภาพคอมโพเนนต์นอกจากจะใช้ในการแสดงโครงสร้างและส่วนประกอบของซอฟต์แวร์แล้ว ยังใช้ในการนำเสนอสถาปัตยกรรมของซอฟต์แวร์ด้วย ทั้งนี้เนื่องจากแผนภาพคอมโพเนนต์นี้จะมีการนำเสนอส่วนประกอบของซอฟต์แวร์ในระดับสูงขึ้นมา ทำให้ผู้พัฒนาสามารถทำความเข้าใจกับสถาปัตยกรรมของซอฟต์แวร์ได้ง่ายกว่า ดังรูป 2.17.



รูปที่ 2.17. ตัวอย่างการออกแบบสถาปัตยกรรมของซอฟต์แวร์เป็นแบบระดับชั้น

2.1.3.6. แผนภาพการติดตั้งใช้งาน (Deployment diagram)

เป็นแผนภาพที่นำเสนอส่วนประกอบของระบบในส่วนที่เป็นฮาร์ดแวร์ รวมถึงการเชื่อมต่อระหว่างกันด้วย สำหรับส่วนประกอบที่สำคัญของแผนภาพการติดตั้งใช้งานนั้นก็คือนode หน่วยประมวลผลหรือโนด (node) และการเชื่อมต่อระหว่างโนด สำหรับโนดนั้นจะเป็นที่บ่งบอกถึง อุปกรณ์ที่มีความสามารถในการประมวลผลที่มีอยู่ในระบบ ไม่ว่าจะเป็นคอมพิวเตอร์ เซิร์ฟเวอร์ และอาจจะรวมถึงอุปกรณ์ต่อพ่วง ดังเช่น พรินเตอร์ หรือสแกนเนอร์ ดังรูป 2.18.

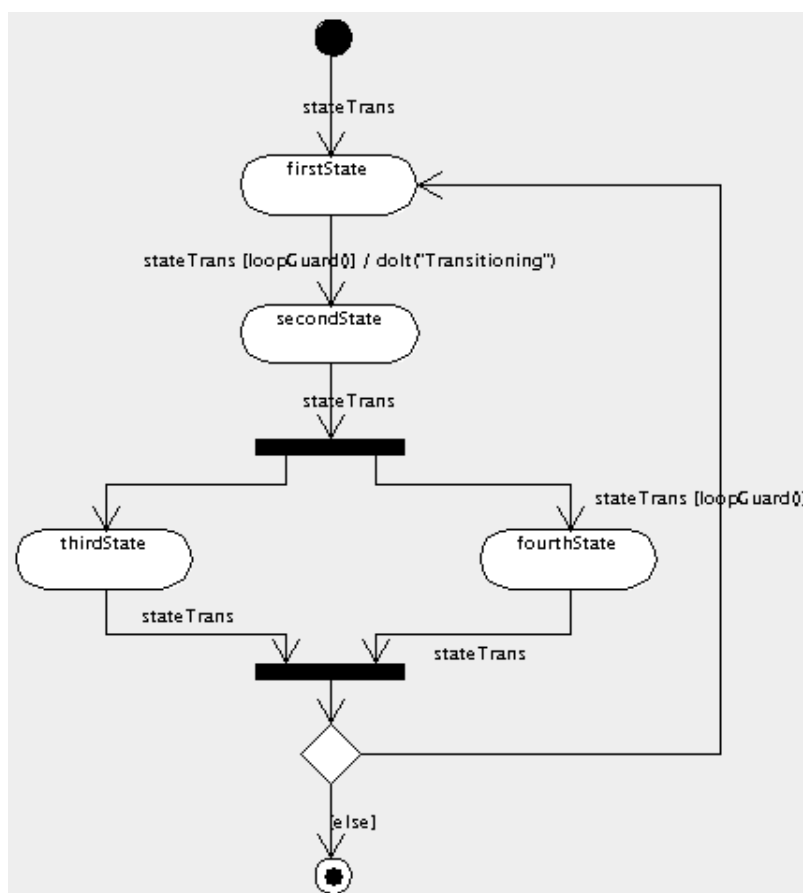


รูปที่ 2.18. ตัวอย่างแผนภาพการติดตั้งการใช้งาน

2.1.3.7. แผนภาพกิจกรรม (Activity diagram)

เป็นแผนภาพที่ใช้ในการแสดงพฤติกรรมของซอฟต์แวร์ในแง่ของลำดับการทำงาน เราอาจจะกล่าวได้ว่าแผนภาพกิจกรรมในโมเดลยูเอ็มแอลนี้เป็นแผนภาพโฟลว์ชาร์ต(Flow chart) แบบหนึ่ง ที่ได้มีการขยายเพิ่มคุณสมบัติขึ้นเพื่อให้เหมาะสมกับการโมเดลซอฟต์แวร์ในปัจจุบัน

โดยส่วนประกอบที่สำคัญในแผนภาพกิจกรรมนั้นก็คือ จุดเริ่มต้น, จุดจบ, กิจกรรม, เงื่อนไข และเส้นทางแสดงลำดับการทำงาน สำหรับการสร้างแผนภาพกิจกรรมนั้นจะคล้ายกับแผนภาพโฟลว์ชาร์ตมาก ดังนั้นผู้ที่คุ้นเคยกับแผนภาพโฟลว์ชาร์ตอยู่แล้วก็สามารถวาดแผนภาพกิจกรรมได้โดยใช้หลักการเดียวกัน ดังรูป 2.19.



รูปที่ 2.19. ตัวอย่างแผนภาพกิจกรรม

2.1.4. กระบวนการทางวิศวกรรมซอฟต์แวร์อาร์ยูพี (Rational Unified Process)

เป็นกระบวนการทางวิศวกรรมซอฟต์แวร์ที่พัฒนาโดยใช้ยูสเคส เป็นตัวขับเคลื่อน (Use Case Driven Development Approach) เป็นการพัฒนาโดยมีสถาปัตยกรรมเป็นศูนย์กลาง (Architecture Centric Development Approach) และมีการพัฒนาเป็นรอบและการพัฒนาแบบก้าวหน้า (Iterative and Incremental) ซึ่งเป็นคุณสมบัติที่ดีในการพัฒนาซอฟต์แวร์ โดยขั้นตอนการทำงานหลักๆนั้นจะมีดังต่อไปนี้คือ

- ขั้นตอนการทำงานสำหรับการโมเดลทางธุรกิจ (Business modeling workflow)
- ขั้นตอนการทำงานที่เกี่ยวข้องกับความต้องการ (Requirements workflow)
- ขั้นตอนการทำงานสำหรับการวิเคราะห์และออกแบบ (Analysis & Design workflow)
- ขั้นตอนการทำงาน ในการพัฒนาและสร้าง(Implementation workflow)
- ขั้นตอนการทำงานในการทดสอบ (Test workflow)
- ขั้นตอนการทำงานในการติดตั้งและใช้งาน (Deployment workflow)

2.1.4.1. ขั้นตอนการทำงานสำหรับการโมเดลทางธุรกิจ (Business modeling workflow)

คือการกำหนดขั้นตอนการทำงานร่วมกันระหว่างทีมพัฒนาทางธุรกิจกับซอฟต์แวร์ เพื่อให้เกิดความสอดคล้องในการพัฒนาและตรงกับความต้องการในการดำเนินธุรกิจมากที่สุด โดยในขั้นตอนการทำงานของโมเดลธุรกิจนั้นจะเกี่ยวข้องกับการจัดทำเอกสาร โดยใช้สิ่งที่เรียกว่า ยูสเคสทางธุรกิจ (Business Use case) ซึ่งโมเดลนี้จะช่วยเพิ่มความเข้าใจระหว่างลูกค้ากับทีมพัฒนาในกระบวนการทางธุรกิจที่จำเป็นต้องได้รับการสร้างขึ้นเป็นซอฟต์แวร์ให้ดีขึ้น ส่วนยูสเคสทางธุรกิจจะถูกวิเคราะห์เพื่อให้ทีมพัฒนาเข้าใจว่าซอฟต์แวร์จะถูกนำไปใช้เพื่อการสนับสนุนการทำงานในธุรกิจอย่างไรซึ่งเราเรียกเอกสารนี้ว่าออบเจกต์โมเดลสำหรับธุรกิจ (Business Object-Model) อย่างไรก็ตามไม่จำเป็นที่โครงสร้างการพัฒนาซอฟต์แวร์ทุกโครงการจะต้องดำเนินการขั้นตอนการทำงานเพื่อสร้างโมเดลทางธุรกิจ โดยขึ้นอยู่กับรูปแบบของซอฟต์แวร์ และองค์กรที่จะใช้งานซอฟต์แวร์

2.1.4.2. ขั้นตอนการทำงานที่เกี่ยวข้องกับความต้องการ (Requirements workflow)

เป็นขั้นตอนที่จะช่วยให้ทีมพัฒนาทราบว่าอะไรคือสิ่งที่ระบบซอฟต์แวร์ต้องทำ อีกทั้งยังช่วยให้ทีมพัฒนากับลูกค้าสามารถทำความเข้าใจในรายละเอียดของซอฟต์แวร์ที่จะพัฒนาขึ้น ในขั้นตอนนี้ทีมพัฒนาจะต้องค้นหา จัดโครงสร้าง และบันทึกความต้องการที่เป็นฟังก์ชันการทำงาน และเงื่อนไขต่างๆ ไว้เป็นเอกสารเพื่อการติดตามและการนำกลับมาใช้เมื่อจำเป็น

ในขั้นตอนเกี่ยวกับความต้องการ จะมีการสร้างเอกสารวิสัยทัศน์ (Vision document) ขึ้น มีการกำหนดแอกเตอร์ซึ่งจะหมายถึงผู้ใช้ซอฟต์แวร์และระบบอื่นๆ ที่จะต้องติดต่อซอฟต์แวร์ที่จะพัฒนาขึ้น มีการกำหนดยูสเคสซึ่งจะนำเสนอถึงพฤติกรรมที่ซอฟต์แวร์จะทำงาน

หลังจากที่ได้ทำการกำหนดยูสเคสแล้วขั้นต่อมาทีมพัฒนาจะทำการกำหนดรายละเอียดของแต่ละยูสเคส คำอธิบายยูสเคสจะแสดงให้เห็นว่าซอฟต์แวร์จะติดต่อกับแอสเตอร์อย่างไรที่และขั้นตอน ส่วนความต้องการที่มาได้เป็นฟังก์ชันการทำงานนั้นจะได้รับการกำหนดและบันทึกไว้ด้วย

2.1.4.3. ขั้นตอนการทำงานสำหรับการวิเคราะห์และออกแบบ (Analysis & Design workflow)

วัตถุประสงค์ของขั้นตอนการทำงานสำหรับการวิเคราะห์และออกแบบนั้นก็เพื่อแสดงให้เห็นว่าซอฟต์แวร์จะทำงานอย่างไรซึ่งรายละเอียดและข้อมูลที่ได้ถูกนำไปใช้ในขั้นตอนของการสร้างต่อไป และผลของขั้นตอนในการวิเคราะห์และออกแบบนี้จะได้เป็น โมเดล ซึ่งเรียกว่า โมเดลการออกแบบที่จะเป็นต้นแบบในการเขียนโค้ด

โมเดลการออกแบบนั้นประกอบไปด้วยคลาสที่ได้มีการจัดโครงสร้างให้อยู่ร่วมกันเป็นแพคเกจ (Package) หรือระบบย่อย (Subsystem) และกิจกรรมในการออกแบบนั้นก็จะเน้นที่แผนภาพสัญลักษณ์ที่จะแสดงถึงสถาปัตยกรรมของซอฟต์แวร์ ซึ่งจากสถาปัตยกรรมนี้จะทำให้ทีมพัฒนาสามารถมองเห็นรายละเอียดที่จะต้องเพิ่มหรือเสริมขึ้นมาเพื่อให้ซอฟต์แวร์สามารถทำงานได้ตามที่ผู้ใช้ต้องการ

2.1.4.3. ขั้นตอนการทำงาน ในการพัฒนาและสร้าง (Implementation workflow)

วัตถุประสงค์ของขั้นตอนการทำงาน ในการพัฒนาและสร้างนั้น ก็เพื่อกำหนดโครงสร้างของโค้ด ในมุมมองของการสร้างระบบย่อยในระดับต่างๆของซอฟต์แวร์ , เพื่อสร้างคลาสและออบเจกต์ของ คอมโพเนนท์, เพื่อทดสอบคอมโพเนนท์ในระดับน้อยย่อย, เพื่อทำการรวมส่วนประกอบที่ได้รับการพัฒนาของนักพัฒนาแต่ละคนหรือแต่ละทีมเข้ามาเป็นซอฟต์แวร์ที่สามารถทำงานได้

ในอาร์ยูพี(Ration Unified Process) คอมโพเนนท์จะถูกกำหนดโครงสร้างให้อยู่ร่วมกันในระบบย่อยของการพัฒนาและสร้าง (Implementation Subsystem) ในการบริหารโครงการระดับย่อยที่ถูกสร้างขึ้นเหล่านี้ควรจะถูกรวบรวมโดยโครงสร้างใดเรีกทอริทั้งนี้เพื่อให้มีการจัดโครงสร้างที่ง่ายต่อการใช้งานและแก้ไขรวมทั้งยังสามารถใช้ในการเก็บข้อมูลที่มีความสำคัญต่อการจัดการได้ด้วย

2.1.4.3. ขั้นตอนการทำงานในการทดสอบ (Test workflow)

เนื่องจากกระบวนการ อาร์ยูพี(Ration Unified Process) นั้นจะเป็นกระบวนการที่มีการพัฒนาเป็นรอบๆ เพราะฉะนั้นในแต่ละช่วงของการพัฒนางี้จะถูกทำการทดสอบอยู่เป็นระยะ โดยในกระบวนการของการทดสอบนั้น จะประกอบไปด้วย การวางแผน การออกแบบ การสร้าง และการลงมือทดสอบและประเมินผล นอกจากนี้แล้วในกระบวนการยังอธิบายว่าทีมพัฒนาจะใช้

เครื่องมือในการทดสอบซอฟต์แวร์เมื่อไหร่และอย่างไร ซึ่งเครื่องมือในการทดสอบเป็นสิ่งสำคัญในกระบวนการพัฒนาแบบเป็นรอบ

2.1.4.4 ขั้นตอนการทำงานในการติดตั้งและใช้งาน (Deployment workflow)

วัตถุประสงค์ของขั้นตอนการทำงานเพื่อติดตั้งและใช้งานนั้นก็เพื่อให้ผลิตภัณฑ์ซอฟต์แวร์สามารถทำงานได้ถูกต้องและผู้ใช้พึงพอใจ โดยในขั้นตอนการติดตั้งและการใช้งานนี้จะประกอบด้วยกิจกรรมย่อยต่างๆ เช่น การเปิดตัวซอฟต์แวร์ต่อผู้ใช้ที่อยู่ภายนอกทีมพัฒนา, การจัดหน้าของตัวซอฟต์แวร์เพื่อเตรียมส่งมอบ, การเผยแพร่ซอฟต์แวร์ การให้ความช่วยเหลือต่อผู้ใช้ในการใช้งานซอฟต์แวร์ บางครั้งขั้นตอนของการทดสอบจะประกอบไปด้วยกิจกรรมในการวางแผนและทดสอบซอฟต์แวร์ในขั้นเบต้า, การทำการย้ายซอฟต์แวร์และข้อมูลที่มีอยู่เพื่อให้พร้อมกับซอฟต์แวร์ใหม่ หรือทำการตกลงเพื่อรับมอบระบบจากลูกค้า

2.1.5 รูปแบบสถาปัตยกรรมของเว็บแอปพลิเคชัน (Web Application Architecture Pattern)

ในการพัฒนาซอฟต์แวร์ทั่วไปนั้น ผู้พัฒนามีทางเลือกในการกำหนดและเลือกสถาปัตยกรรมสำหรับซอฟต์แวร์ได้อย่างหลากหลาย เช่น รูปแบบสถาปัตยกรรมสำหรับควบคุมรูปแบบสถาปัตยกรรมความปลอดภัย แต่สำหรับการพัฒนาเว็บแอปพลิเคชันนั้น ทางเลือกของสถาปัตยกรรมที่จะใช้นั้นจะขึ้นอยู่กับเทคโนโลยีและผลิตภัณฑ์ที่ใช้ และด้วยคุณสมบัติของเว็บแอปพลิเคชันที่ให้ผู้ใช้งานสามารถประมวลผลและแลกเปลี่ยนข้อมูลกับเว็บเซิร์ฟเวอร์ได้ ดังนั้นเราจึงสามารถที่จะกำหนดส่วนประกอบที่สำคัญของเว็บแอปพลิเคชันได้ คือ โปรแกรมเว็บเบราว์เซอร์ที่ไคลเอนต์ เว็บเซิร์ฟเวอร์ แอปพลิเคชันเซิร์ฟเวอร์ และอาจจะประกอบด้วย เซิร์ฟเวอร์ฐานข้อมูลเพิ่มขึ้นมาในระบบงานที่เกี่ยวข้องกับการเก็บข้อมูลเป็นจำนวนมาก

โดยรูปแบบของสถาปัตยกรรมที่เหมาะสมสำหรับการประยุกต์ใช้ในเว็บแอปพลิเคชันนั้นสามารถแบ่งออกได้เป็น 3 รูปแบบ คือ ทินเว็บไคลเอนต์ (Thin Web Client) ทริกเว็บไคลเอนต์ (Thick Web Client) และเว็บเดลิเวอรี่ไคลเอนต์ (Web Delivery Client) โดยรูปแบบสถาปัตยกรรมทั้ง 3 รูปแบบนี้จะเกี่ยวข้องกับตำแหน่งของส่วนประกอบที่เกี่ยวข้องกับการประมวลผลเป็นหลัก ซึ่งรูปแบบแต่ละแบบก็มีข้อดีข้อเสียต่างกันไป

2.1.5.1 รูปแบบสถาปัตยกรรมทินเว็บไคลเอนต์ (Thin Web Client)

เป็นรูปแบบสถาปัตยกรรมเว็บแอปพลิเคชันที่ส่วนใหญ่เลือก ทั้งนี้เป็นรูปแบบสถาปัตยกรรมที่ไคลเอนต์ไม่จำเป็นต้องมีความสามารถมากนัก เนื่องจากผู้ใช้งานใช้ไคลเอนต์เป็นเพียงช่องทางสำหรับรับและส่งข้อมูลเท่านั้น ส่วนการประมวลผลทั้งหมดจะอยู่ที่เซิร์ฟเวอร์

2.1.5.2 รูปแบบสถาปัตยกรรมทริกเว็บไคลเอนต์ (Thick Web Client)

เป็นรูปแบบสถาปัตยกรรมที่เริ่มมีการประมวลผลบางส่วนกระทำที่ฝั่งไคลเอนต์ ทั้งนี้ไคลเอนต์จะต้องมีความสามารถในการรันภาษาสคริปต์หรือออบเจกต์ของจาวา และ แอคทีฟเอ็กซ์ (ActiveX) สำหรับการสื่อสารระหว่างไคลเอนต์กับเซิร์ฟเวอร์นั้นยังคงทำผ่าน โพรโตคอลเอชทีทีพี (HTTP) ปกติ

2.1.5.3 รูปแบบสถาปัตยกรรมเว็บเดลิเวอรี่ไคลเอนต์ (Web Delivery Client)

เป็นรูปแบบสถาปัตยกรรมที่มีการใช้โพรโตคอลเอชทีทีพี เพียงการสื่อสารในครั้งแรกเท่านั้นแต่การสื่อสารหลังจากนั้นที่เกี่ยวข้องกับการประมวลผลอาจจะกระทำผ่านโพรโตคอลอื่น เช่น ไอไอโอพี (IIOP) หรือ อาร์เอ็มไอ (RMI) ของจาวา รูปแบบการทำงานแบบนี้จะเป็นการทำงานของออบเจกต์แบบกระจาย โดยโปรแกรมเว็บเบราว์เซอร์จะทำหน้าที่เป็นช่องทางเพื่อให้ออบเจกต์ทำงานเท่านั้น

2.2. เทคโนโลยีคอตเน็ต (.NET)

2.2.1. ภาพรวมของเทคโนโลยีคอตเน็ต (.NET)

คอตเน็ต (.NET) เป็นกลยุทธ์ของทางบริษัทไมโครซอฟท์ (Microsoft) ที่คิดจะทำการเชื่อมต่อข้อมูล, ผู้คน, ระบบและอุปกรณ์ผ่านซอฟต์แวร์เว็บเซอร์วิส (Web Service) ทำให้ทีมงานสามารถสื่อสารและทำงานร่วมกันได้อย่างมีประสิทธิภาพ

เทคโนโลยีคอตเน็ต .NET ถูกรวมอยู่ในผลิตภัณฑ์ทั้งหมดของไมโครซอฟท์ (Microsoft) เพื่อเพิ่มความสามารถในการสร้าง, การจัดการ และติดต่อได้อย่างรวดเร็ว เพิ่มแนวทางด้านความปลอดภัย (security-enhanced solutions) ด้วยเว็บเซอร์วิส Web Service

แนวทางการเชื่อมต่อคอตเน็ต (.NET-connected solutions) เป็นการรวมระบบธุรกิจให้สามารถทำงานได้เร็วขึ้น มีความว่องไว กระชับกระเฉง และช่วยให้เราเข้าถึงการเปลี่ยนแปลงของข้อมูลได้ทุกที่ ทุกเวลา และทุกอุปกรณ์

2.2.2. ประโยชน์ของเทคโนโลยีคอตเน็ต(.NET)

2.2.2.1. ประโยชน์ทางด้านองค์กร (.NET benefits organizations)

คอตเน็ต (.NET) จะลบข้อจำกัดในเรื่องไพรไพรทาลี เทคโนโลยี (proprietary technology) ทำให้มีความยืดหยุ่นสามารถช่วยขยายการติดต่อระหว่างระบบ IT ที่มีในปัจจุบัน เป็นการสร้างกลิ่นลูกใหม่ของเทคโนโลยีคอมพิวเตอร์คอตเน็ต .NET และเว็บเซอร์วิส (Web services) ช่วยองค์กรลดค่าใช้จ่ายในการติดต่อกับระบบ เพิ่มยอดขายโดยช่วยให้พนักงานเข้าถึง

ข้อมูลที่ถูกต้องในเวลาและสถานที่ที่เขาต้องการ ช่วยให้นักพัฒนาสร้างโซลูชัน (solution) ทางธุรกิจได้อย่างรวดเร็วและมีค่าใช้จ่ายต่ำ

2.2.2.2. ประโยชน์ในส่วนบุคคล (.NET benefits individuals)

ช่วยให้ได้รับข้อมูลตามความต้องการของแต่ละบุคคลมากขึ้น ซึ่งสามารถรองรับฮาร์ดแวร์ได้หลากหลาย เช่น Pocket PCs, Smart phones, laptops เป็นต้น ผู้ใช้โมบาย คอมพิวเตอร์ (mobile computing) สามารถรับข้อมูลได้ทุก ๆ ที่

2.2.2.3. ประโยชน์ในด้านการพัฒนา (.NET benefits developers)

มีการรวมเอากลุ่มของ tools เพื่อใช้ในการสร้าง Web services ด้วย Visual Studio .NET และ .NET Framework ซึ่งพัฒนาได้ในหลาย platform เป็นหนทางช่วยผู้พัฒนาให้พัฒนา Web service ได้ดี, รวดเร็วและมีประสิทธิภาพมากที่สุด

2.2.3. ผลกระทบและเทคโนโลยีของ Microsoft ในปัจจุบันที่มีการใช้ .NET

2.2.3.1. .NET Passport

เป็น Web-based service ที่ออกแบบมาเพื่อใช้สำหรับการล็อกอิน (login) เข้าสู่เว็บไซต์ต่างๆ ได้อย่างง่ายดายและรวดเร็ว โดยตัวพาสปอร์ต (passport) นี้ ทำให้เว็บไซต์ที่รองรับ .NET Passport สามารถให้สิทธิ์แก่ผู้ใช้จาก single set of sign-in credentials ทำให้ผู้ใช้ไม่จำเป็นต้องจดจำชื่อผู้ใช้ (username) และรหัสผ่าน (password) จำนวนมาก

ไมโครซอฟต์ดอทเน็ต (Microsoft .NET) มีทุก ๆ สิ่งที่เป็นในการพัฒนา service-based IT architecture:

- servers to host Web services
- development tools เพื่อใช้สร้าง Web services
- applications เพื่อที่จะเรียกใช้เว็บเซอร์วิส Web services เทคโนโลยีดอทเน็ต .NET

มีการรองรับไว้ในผลิตภัณฑ์ตระกูลไมโครซอฟต์ (Microsoft) ทั้งหมด เช่น Windows Server System, Windows XP desktop operating system และ Microsoft Office System

2.2.3.2. ตัวอย่างองค์กรที่มีการใช้เทคโนโลยี .NET

องค์กรทั่วโลกมีการนำเทคโนโลยีดอทเน็ต (.NET) และเว็บเซอร์วิส (Web service) ไปใช้เพื่อช่วยเชื่อมต่อธุรกิจและทำให้การสื่อสาร การทำงานร่วมกันมีประสิทธิภาพมากยิ่งขึ้น ตัวอย่างเช่น Honeywell, GlaxoSmithKline, Sony, Dollar Rent A Car, Farmers Insurance และ the United States Postal Service ที่เป็นองค์กรที่มีการใช้เทคโนโลยี .NET ในปัจจุบัน

2.3. เทคโนโลยีเว็บเซอร์วิส (Web Service)

2.3.1. ภาพรวมของเทคโนโลยี Web Service

เว็บเซอร์วิส (Web Service) คือ วิธีการนำกลับมาใช้ของแอปพลิเคชัน (reusable application) ที่ช่วยให้แอปพลิเคชัน ทำงานร่วมกันได้ด้วยการแลกเปลี่ยนเมสเสจ (message) ผ่านทางโปรโตคอลมาตรฐานของการติดต่อกันในอินเทอร์เน็ต (standard internet communication protocol) เช่น

- XML (Extensible Markup Language)
- SOAP (Simple Object Access Protocol)
- WSDL (Web Services Description Language)

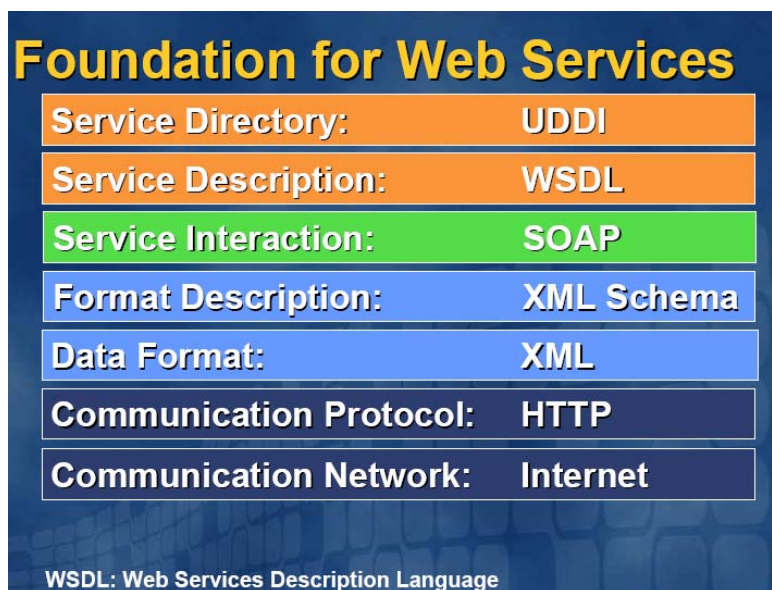
โดยโปรโตคอลเหล่านี้จะทำให้คอมพิวเตอร์สามารถทำงานร่วมกันข้ามภาษาและข้ามแพลตฟอร์มได้ เป็นเว็บแอปพลิเคชัน (Web application) ที่ไม่มียูสเซอร์อินเทอร์เฟซ (user interface)

Web services เป็นการปฏิวัติวิธีการสื่อสารกันระหว่างแอปพลิเคชัน (applications) แลกเปลี่ยนข้อมูลผ่านอินเทอร์เน็ต (internet) โดยการใช้ universal data format ทำให้เว็บเซอร์วิส (Web services) สามารถสื่อสารข้ามแพลตฟอร์ม (platform) และระบบปฏิบัติการ และไม่ขึ้นกับภาษาที่ใช้เขียนแอปพลิเคชัน (applications)

แต่ละ เว็บเซอร์วิส (Web services) จะเป็น discrete unit of code ที่รองรับงานที่จำกัด แม้ว่า เว็บเซอร์วิส (Web services) จะเป็นอิสระต่อกัน แต่มันก็สามารถรวมตัวกันเป็นกลุ่มแบบหลวมๆ เพื่อที่จะทำงานเฉพาะอย่างได้

2.3.2. โครงสร้างชั้นพื้นฐานของเทคโนโลยีเว็บเซอร์วิส (Web services)

โครงสร้างพื้นฐานของเทคโนโลยี Web Service ประกอบไปด้วยส่วนต่างๆ ดังรูป 2.20.



รูปที่ 2.20. แสดงโครงสร้างพื้นฐานของเทคโนโลยี Web Service

2.3.2.1. เอ็กซ์เอ็มแอล XML (Extensible Markup Language)

เอ็กซ์เอ็มแอล (Extensible Markup Language) คือมาตรฐานภาษาอธิบายข้อมูลที่ใช้กันทั่วโลกสำหรับการใช้ข้อมูลทางธุรกิจร่วมกันบนเว็บ โดยไม่ต้องกังวลเรื่องโปรแกรม เครื่องข่าย คอมพิวเตอร์ โครงสร้างข้อมูลและระบบปฏิบัติการที่เข้ากันไม่ได้ โดยที่ เอ็กซ์เอ็มแอล (Extensible Markup Language) ทำให้สามารถใช้ข้อมูลร่วมกันได้ง่ายขึ้น

เอ็กซ์เอ็มแอล (Extensible Markup Language) ใช้แท็ก (tag) มาตรฐานในการระบุโครงสร้างและเนื้อหาของแฟ้ม การใช้แท็กเอ็กซ์เอ็มแอล (tag XML) เหมือนกันในทุกแฟ้มจะช่วยให้คุณสามารถทำดัชนี ค้นหา รวม และนำข้อมูลที่ใช้ข้อความพื้นฐานมาใช้ใหม่ได้อย่างมีประสิทธิภาพ และเนื่องจากเอ็กซ์เอ็มแอล (Extensible Markup Language) ใช้ข้อความพื้นฐานและไม่จำกัดสำหรับภาษาคอมพิวเตอร์ส่วนตัวเท่านั้น จึงสามารถแลกเปลี่ยนข้อมูลระหว่างระบบที่โดยปกติไม่เข้ากันได้

2.3.2.2. เอสโอเอพี SOAP (Simple Object Access Protocol)

เอสโอเอพี (SOAP) คือ XML-based messaging technology เป็นมาตรฐานสำหรับเมสเสจของเอ็กซ์เอ็มแอล (XML messaging) ถูกพัฒนาให้เป็นมาตรฐานโดยคับบลิวทรีซี (W3C) ซึ่งกำหนด rule ที่สำคัญบนเว็บเซอร์วิส (Web services), วิธีการการรวม web service เข้าไปใน application และการสื่อสารระหว่างกัน

2.3.2.3. ดับบลิวเอสดีแอล WSDL (Web Services Description Language)

ดับบลิวเอสดีแอล (WSDL (Whiz-Dull)) คือ XML-based service description ที่อธิบายว่าหากต้องการใช้ เว็บเซอร์วิส (Web services) ต้องทำการติดต่ออย่างไร ต้องมีรูปแบบของเมสเสจ

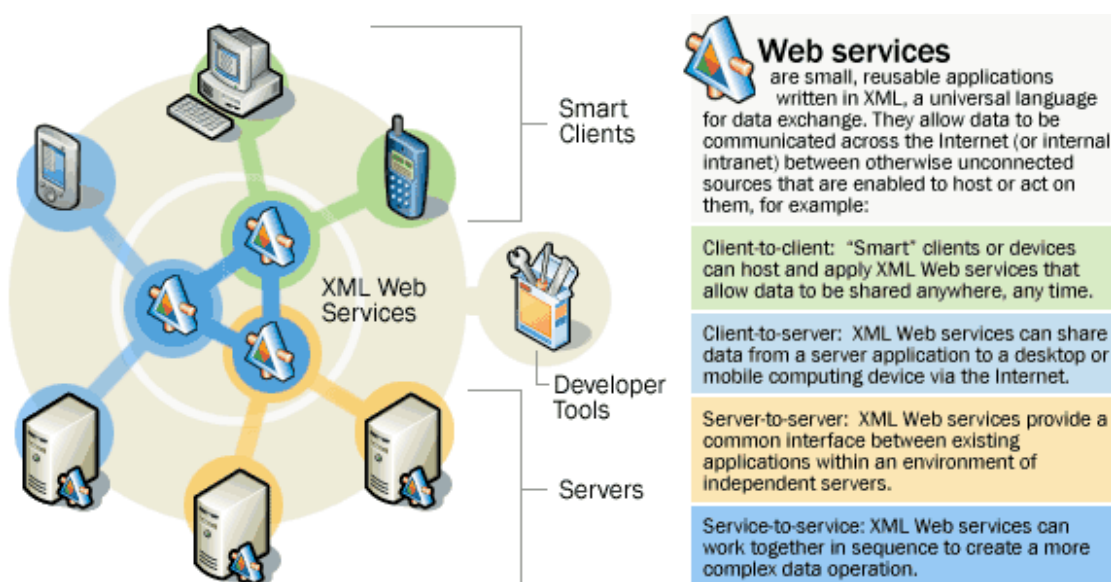
(message formats) เป็นอย่างไร เป็นตัวอธิบาย public interface สำหรับเว็บเซอร์วิส (Web services) เมื่อไคลเอ็นต์ ทำการติดต่อมายังเว็บเซอร์วิส (Web services) สามารถอ่านฉบับลิวเอสดีแอล (WSDL) เพื่อที่จะรับรู้ว่ามีฟังก์ชัน (function) ใดบ้างที่ให้บริการบน (server)

2.3.2.4. ยูดีดีไอ UDDI (Universal Description, Discovery, and Integration)

ยูดีดีไอ UDDI เป็น public registry ไม่เสียค่าใช้จ่าย เป็นที่ๆ ผู้คนสามารถเผยแพร่และสอบถามข้อมูลของเว็บเซอร์วิส (Web services) ได้ อธิบายวิธีการมาตรฐานในการเก็บและรับข้อมูลเกี่ยวกับเว็บเซอร์วิส (Web services)

2.3.3. ลักษณะการเชื่อมต่อของเทคโนโลยีเว็บเซอร์วิส (Web services)

การเชื่อมต่อของเว็บเซอร์วิส (Web services) มีได้หลากหลายแบบ ดังรูป 2.21.



รูปที่ 2.21. แสดงลักษณะการเชื่อมต่อของเทคโนโลยี Web Service

2.3.3.1. ไคลเอ็นต์กับไคลเอ็นต์ (Client-to-client)

สมาร์ตไคลเอ็นต์ Smart clients สามารถเป็นโฮสต์ (host) และประยุกต์ใช้เว็บเซอร์วิส (Web services) เพื่อทำการแชร์ข้อมูลได้ทุกที่ ทุกเวลา

สมาร์ตไคลเอ็นต์ (Smart client) คือไคลเอ็นต์แอปพลิเคชัน (client applications) ที่มีการใช้บริการเว็บเซอร์วิส (Web services) โดยจะอยู่บนฮาร์ดแวร์ของผู้ใช้ เช่น desktop PCs, laptops, Pocket PCs และ Smart phones มันสามารถทำการจัดการ ปรับใช้ได้ง่ายโดยการใช้ทรัพยากรบนอุปกรณ์ และสามารถติดต่อกับ distributed data sources ได้

สมาร์ตไคลเอนต์แอปพลิเคชัน (Smart client applications) สามารถทำงานได้ไม่ว่าจะมีการติดต่อกับอินเทอร์เน็ตหรือไม่ ตัวอย่างเช่น Microsoft Money , Microsoft Outlook ต่างกับ "browser-based" applications เช่น Amazon.Com หรือ eBay.com

2.3.3.2. ไคลเอนต์กับเซิร์ฟเวอร์ (Client-to-Server)

เอ็กซ์เอ็มแอลเว็บเซอร์วิส (XML Web services) สามารถแชร์ข้อมูลจากเซิร์ฟเวอร์แอปพลิเคชัน (server application) ไปยังเครื่องคอมพิวเตอร์ desktop หรือ mobile computing device ผ่านอินเทอร์เน็ตได้

2.3.3.3. เซิร์ฟเวอร์กับเซิร์ฟเวอร์ (Server-to-Server)

เอ็กซ์เอ็มแอลเว็บเซอร์วิส (XML Web services) ให้บริการด้าน common interface ระหว่าง application ที่มีการรันอยู่ตามสภาพแวดล้อมของแต่ละ server

2.3.3.4. เซอร์วิสกับเซอร์วิส (Service-to-service)

เอ็กซ์เอ็มแอลเว็บเซอร์วิส (XML Web services) สามารถทำงานร่วมกันได้เป็นลำดับเพื่อที่จะทำการสร้าง operation ที่ซับซ้อนขึ้น

2.3.4. เทคโนโลยีเซอร์วิส (Web services) ในมุมมองทางธุรกิจ

ผู้นำกลุ่มธุรกิจกล่าวว่าเซอร์วิส (Web services) คือวิธีที่ช่วยให้ธุรกิจติดต่อกับลูกค้า, พันธมิตรธุรกิจ, ผู้ผลิตและลูกค้าได้อย่างมีประสิทธิภาพ เหมือนเป็นการปลดล็อกข้อมูลเพราะข้อมูลจะวิ่งไปได้ทุกๆ ที่ หากมีผู้ต้องการ ลดเวลาในการพัฒนาและค่าใช้จ่ายสำหรับโครงการใหม่ จากมุมมองทางธุรกิจเว็บเซอร์วิส (Web services) ไม่เพียงแค่เชื่อมต่อระบบ แต่มันยังช่วยเชื่อมต่อผู้คนเข้ากับข้อมูลพวกเค้าต้องการ ด้วย software applications ที่พวกเค้าใช้อยู่

ผลิตภัณฑ์ที่ไม่โครซอฟท์ (Microsoft) ได้เสนอเพื่อช่วยให้องค์กรและแต่ละบุคคลได้รับประโยชน์จาก **Web service-based connectivity** มีตัวอย่างเช่น

- Microsoft Visual Studio .NET 2003 ซึ่งเป็น developer tools
- Windows Server System ซึ่งเป็น hosts Web services

และ desktop applications ที่เรากำลังใช้อย่าง

- Microsoft Office System ซึ่งเป็นระบบที่จะเรียกใช้ Web services.

2.3.5. เปรียบเทียบเว็บไซต์กับแอ็พพลิเคชันเว็บเซอร์วิส (XML Web service)

เว็บไซต์มักเกี่ยวข้องกับการนำเสนอข้อมูลให้กับผู้ใช้ โดยเป็นเครื่องมือในการสื่อสารสำหรับเซิร์ฟเวอร์ไปยังผู้ใช้ แต่ในทางตรงกันข้าม บริการแอ็พพลิเคชันเว็บ (XML Web) เป็นวิธีที่แอปพลิเคชันจะสามารถโต้ตอบกับแอปพลิเคชันอื่นได้โดยตรง ไม่ว่าจะเป็นแอปพลิเคชันที่อยู่บนเซิร์ฟเวอร์เดียวกัน หรืออยู่บนระบบที่ห่างไกลออกไป ก็สามารถสื่อสารกันได้โดยใช้อินเทอร์เน็ตและระบบข่าวสารแอ็พพลิเคชัน (XML) และเอสโอเอพี (SOAP)

พิจารณาจากตัวอย่างง่ายๆ คือ หากคุณมีระบบสินค้าคงคลัง แต่ไม่ได้เชื่อมโยงเข้ากับระบบใดเลย ก็เท่ากับว่าคุณค่าของระบบนั้นมีไม่มาก แม้จะสามารถติดตามสินค้าคงคลังได้ แต่ต้องดำเนินการเป็นจำนวนมาก และสิ่งที่ระบบสามารถทำงานได้ด้วยตัวเองโดยไม่ต้องเชื่อมโยงกับระบบอื่นนั้นก็ค่อนข้างจำกัด เพราะทุกรายการสินค้าที่ขายไม่เพียงแต่จะต้องป้อนเข้าสู่ระบบสินค้าคงคลังเท่านั้น แต่ต้องกระจายไปสู่ระบบบัญชีและบันทึกในบัญชีของลูกค้าและยังจำเป็นต้องทำบันทึกเพื่อเตรียมส่งสินค้าเพิ่มจากซัพพลายเออร์ (supplier) อีกด้วย สัดส่วนระหว่างต้นทุนและประโยชน์ที่ได้รับนั้นดูแล้วอาจไม่น่าดึงดูดใจ เนื่องจากสิ่งที่ได้รับจากระบบนั้นใกล้เคียงกับค่าใช้จ่ายที่ต้องเสียไป

แต่ถ้าคุณทำการเชื่อมโยงระบบนี้เข้ากับระบบบัญชี ประโยชน์ที่ได้รับจะเพิ่มมากขึ้น เพราะเมื่อมีการซื้อ-ขายเกิดขึ้น การทำงานที่สัมพันธ์กันระหว่างสินค้าคงคลังและกระแสเงินสดของคุณ จะเกิดขึ้นต่อเนื่องในขั้นตอนเดียว และเมื่อเพิ่มการเชื่อมโยงเข้าสู่ระบบจัดการคลังสินค้า, ระบบสั่งซื้อสินค้าของลูกค้า, ระบบสั่งซื้อสินค้าจาก supplier และเชื่อมต่อกับบริษัทจัดส่งสินค้า ซึ่งเมื่อถึงจุดนี้ ระบบจัดการสินค้าคงคลังของคุณจะมีคุณประโยชน์อย่างแท้จริงคือ คุณสามารถจัดการธุรกิจของคุณได้อย่างครบถ้วน โดยทำงานกับแต่ละธุรกรรมเพียงครั้งเดียวเท่านั้น แทนที่จะต้องดำเนินการกับทุกๆ ระบบที่เกี่ยวข้อง ปริมาณงานจึงลดลง และเกิดข้อผิดพลาดน้อยลง

2.3.6. ตัวอย่างผลิตภัณฑ์ของไมโครซอฟท์ Microsoft ที่มีการเรียกใช้แอ็พพลิเคชันเว็บเซอร์วิส (XML Web services)

2.3.6.1. ออฟฟิศ 2003 (Office 2003)

มีการสนับสนุนแอ็พพลิเคชัน (XML) ในออฟฟิศโปรเฟสชันนอล (Office Professional Edition 2003) ซึ่ง

- ช่วยให้บริษัท พัฒนาโซลูชัน (solution) ที่สร้างมาตรฐานให้การสร้างและการเก็บรวบรวมข้อมูลซึ่งอยู่ในรูปแบบไบนารีในคอมพิวเตอร์และในแฟ้มที่ใช้ร่วมกันต่างๆ องค์กรธุรกิจสามารถช่วยลดการซ้ำซ้อนของข้อมูลและงาน
- ช่วยให้พนักงานเข้าถึงและวิเคราะห์ข้อมูลจากระบบธุรกิจที่หลากหลายได้ เช่น โปรแกรมประยุกต์เกี่ยวกับการบริหารสัมพันธ์กับลูกค้า (CRM) ระบบการ

วางแผนทรัพยากรสำหรับองค์กร (ERP) หรือโปรแกรมประยุกต์ในสายงานธุรกิจอื่นๆ

- บริษัทสามารถดึงข้อมูลเอ็กซ์เอ็มแอล (XML) จากแหล่งต่างๆ มาแยกสารฐานข้อมูล และงานนำเสนอได้

วิศวกรสนามนำคอมพิวเตอร์โน้ตบุ๊กที่ติดตั้งอินโฟพาส (InfoPath 2003) ซึ่งเป็นโปรแกรมป้อนข้อมูลโดยใช้ฟอร์มเป็นพื้นฐานมาใช้ในการรวบรวมข้อมูลเกี่ยวกับสถานะของอุปกรณ์ เช่น บั้มหรือเครื่องกำเนิดไฟฟ้า หลังจากทีรวบรวมแล้ว จะกรอกข้อมูลนี้ลงในใบสั่งงานสำหรับการซ่อมบำรุง และสร้างแฟ้มประวัติสำหรับสินทรัพย์แต่ละรายการ

ผู้บริหารด้านการเงินที่ต้องการสร้างรายงานรวมจากหลายแผนกในบริษัทจะส่งออกตัวเลขรายได้จากแต่ละแผนกไปยังกระดาษคำนวณ Excel 2003 ไม่ว่าผู้จัดการแผนกจะเก็บตัวเลขเหล่านี้ไว้ที่ใด การรวมข้อมูลก็จะเป็นไปอย่างราบรื่นเพราะมีการติดตามตำแหน่งข้อมูลด้วยเอ็กซ์เอ็มแอล (XML) อยู่เสมอ

นักวิเคราะห์งานวิจัยใช้ Word 2003 เพื่อสร้างรายงานแบบกำหนดเองเกี่ยวกับโทรศัพท์เคลื่อนที่รุ่นล่าสุดสำหรับลูกค้า นักวิเคราะห์รวบรวมข้อมูลที่มี tag XML จากแหล่งข้อมูลภายในและภายนอกหลายแหล่งลงในแฟ้มเดียว หลังจากทีแฟ้มถูกสร้างขึ้น จะสามารถปรับปรุงเอกสารได้ตลอดเวลาด้วยการเพิ่มข้อมูลในแฟ้มและสร้างรายงานใหม่

2.4. ไมโครซอฟท์ ดอทเน็ตเฟรมเวิร์ก (Microsoft .NET Framework)

2.4.1. ความหมายของเฟรมเวิร์ก (Framework) และแพลตฟอร์ม (Platform)

2.4.1.1. แอปพลิเคชัน เฟรมเวิร์ก (Application Framework)

Application Framework หมายถึง ชุดของ library หรือ class ที่ใช้สำหรับสร้างโครงสร้างมาตรฐานของ Application ของแต่ละ OS โดยมี reusable code จำนวนมากใน framework เพื่อประหยัดเวลาของผู้พัฒนาโปรแกรม ทำให้ไม่ต้องเขียนโค้ดมาตรฐานทุกๆ ครั้งที่จะมีการพัฒนา application ใหม่ๆ

2.4.1.2. แพลตฟอร์ม (Platform)

แพลตฟอร์ม (Platform) หมายถึง ระบบคอมพิวเตอร์ที่แอปพลิเคชันใช้รันอยู่ในปัจจุบัน ซึ่งประกอบไปด้วย ระบบปฏิบัติการ (Operating System) และฮาร์ดแวร์ ระบบปฏิบัติการจะคอยส่งชุดคำสั่งไปยังไมโครโปรเซสเซอร์ให้ประมวลผล โดยระบบปฏิบัติการก็จะถูกออกแบบมาเพื่อใช้กับไมโครโปรเซสเซอร์แต่ละรุ่นตามสถาปัตยกรรม

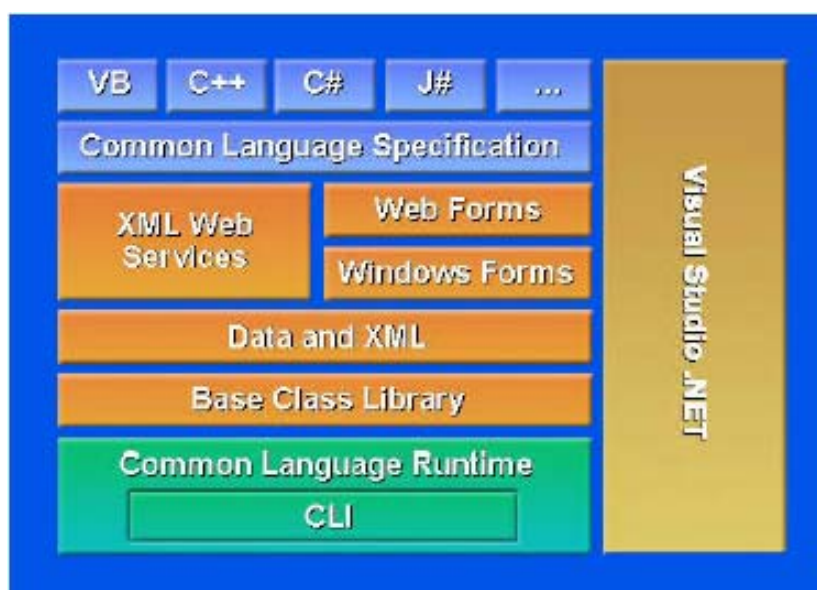
2.4.2. โครงสร้างของดอทเน็ตเฟรมเวิร์ก (.NET Framework)

2.4.2.1. ภาพรวมโครงสร้างของดอทเน็ตเฟรมเวิร์ก (.NET Framework)

ดอทเน็ตเฟรมเวิร์ก (.NET Framework) คือสภาพแวดล้อมในการพัฒนาโปรแกรมด้วยเทคโนโลยี ที่เชื่อมต่อระหว่างระบบปฏิบัติการของเครื่องคอมพิวเตอร์กับ tools ที่เราใช้ในการพัฒนา

เราสามารถพัฒนาโปรแกรมที่ทำงานเสถียรและไม่เปลืองทรัพยากรของระบบได้ โดยแทบไม่ต้องรู้เทคนิคการเขียนโปรแกรมที่เกี่ยวข้องใดๆ เลย ทั้งนี้ก็เพราะ .NET Framework จะคอยจัดการให้เราเอง

ดอทเน็ตเฟรมเวิร์ก (.NET Framework) เป็นคอมโพเนนต์ (component) ภายในวินโดวส์ (Windows) ที่รองรับการสร้างและรันแอปพลิเคชันที่มีการเรียกใช้เอ็กซ์เอ็มแอลเว็บเซอร์วิส (XML Web Services) อย่างสมบูรณ์ ดังรูป 2.22.



รูปที่ 2.22. แสดงโครงสร้างและส่วนประกอบของ .NET Framework

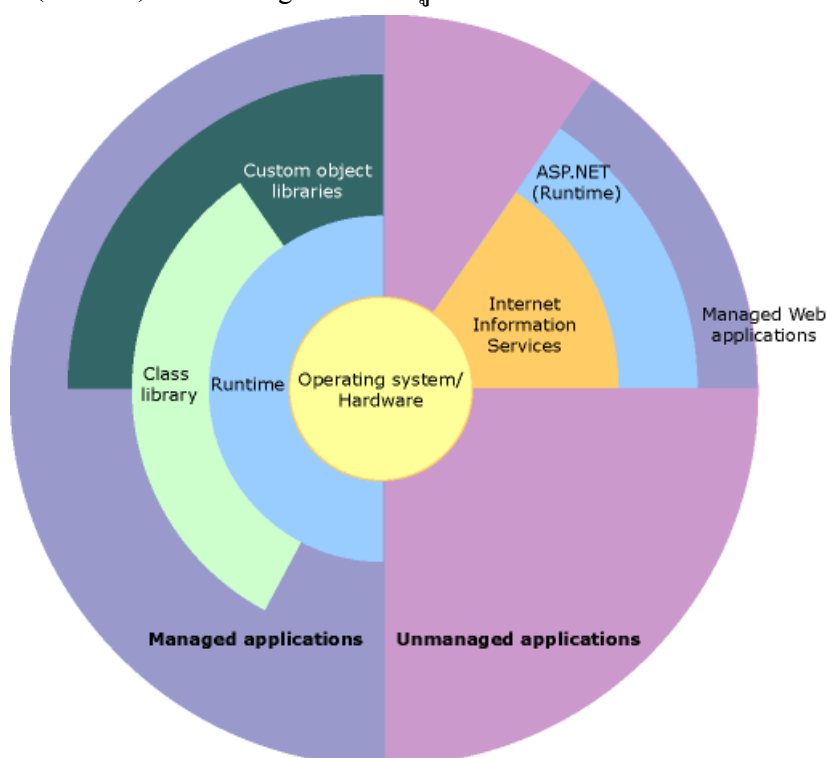
ดอทเน็ตเฟรมเวิร์ก (.NET Framework) ใช้คลาสไลบรารีร่วมกัน โดยสร้างชุดของ API ขึ้นมาให้สามารถทำการสืบทอด (inherit), (error handling) และการดีบัก (debugging) ข้ามภาษาได้ ไม่มีการแบ่งแยกว่า C++ พัฒนาโดยใช้ MFC Java พัฒนาโดยใช้ WFC

ดอทเน็ตเฟรมเวิร์ก 1.1 (.NET Framework 1.1) พัฒนาขึ้นจากดอทเน็ตเฟรมเวิร์ก 1.0 (.NET Framework 1.0) โดยบรรจุความสามารถใหม่ๆ, เสริมการทำงานให้ดีขึ้น และปรับปรุงทางด้านความสามารถในการขยายระบบ, ความน่าเชื่อถือ, เทคโนโลยีด้านความปลอดภัยและประสิทธิภาพ ตัวอย่างเช่น เพื่อวางใจในการทำงานของ application ได้มากขึ้น ผู้บริหารระบบจะ

สามารถกำหนดสิทธิการทำงานให้กับแอปพลิเคชัน เอเอสพีคอตเน็ต และเว็บเซอร์วิส (Web services) ได้ จึงเกิดความยืดหยุ่นมากขึ้น และเพิ่มระดับการป้องกันจากผู้ที่ใช้ที่ประสงค์ร้าย

2.4.2.2. Common language runtime

มีหน้าที่รับผิดชอบเรื่องการจัดการเมโมรี , เริ่ม หยุดคโปรเซส และเทอร์ค (thread) , ควบคุม นโยบายด้านความปลอดภัยและข้อจำกัดที่คอมโพเนนท์หนึ่งๆ จะมีต่อ คอมโพเนนท์อื่นๆ โดย จะเรียกโค้ดที่ถูกกำหนดไว้ในรันทาม (run time) ว่า managed code และเรียกโค้ดที่ไม่ถูกกำหนดไว้ใน รันทาม (run time) ว่า unmanaged code ดังรูป 2.23.



รูปที่ 2.23. แสดงส่วนที่ .NET Framework มีหน้าที่รับผิดชอบ

โปรแกรมคอตเน็ต (.NET) ใดๆ จะถูกคอมไพล์เป็นภาษากลาง (Common Language) ซึ่งมีชื่อเรียกว่า Microsoft Intermediate Language (MSIL หรือ IL) ก่อนเสมอ ทำให้ในทีมพัฒนาเดียวกันสามารถเลือกภาษาที่แตกต่างกันได้ ผู้ที่มารับดูแลโปรแกรมต่อสามารถต่อเติมงานอื่นๆ ได้โดยไม่จำเป็นต้องใช้ภาษาเดิม ก่อนที่ common language runtime จะแปลง IL ให้เป็นภาษาเครื่อง

2.4.2.3. คลาสไลบรารี (Class library)

เป็น Object-oriented class library ที่นำกลับมาใช้ได้ ก่อให้เกิดรูปแบบที่แน่นอนในการเข้าถึงฟังก์ชันต่างๆ สามารถนำไปพัฒนา application ในส่วนที่เป็นมาตรฐานต่างๆ ได้ จึงช่วยลด

ความจำเป็นในการเรียนรู้ ในการเขียนแอปพลิเคชัน แบ่งออกเป็นกลุ่มๆ ตามหน้าที่ที่เกี่ยวข้อง เช่นการจัดการกับ string , การเก็บข้อมูล , การติดต่อกับฐานข้อมูล , การติดต่อไฟล์ เป็นต้น ซึ่งจะลดเวลาในการเรียนรู้ feature ใหม่ๆ ของ .NET Framework

2.5. เอเอสพีคอตเน็ต (ASP.NET)

2.5.1. ภาพรวมของ เอเอสพีคอตเน็ต (ASP.NET)

เป็นเว็บโปรแกรมมิ่งเทคโนโลยี (web programming technology) ถูกสร้างอยู่บนพื้นฐานคลาสของคอตเน็ตเฟรมเวิร์ก (.NET Framework) ในรูปของชุดคอนโทรลและโครงสร้างพื้นฐานที่ง่ายต่อการสร้างเว็บแอปพลิเคชัน (web application) และ เอ็กซ์เอ็มแอลเว็บเซอร์วิส (XML Web services)

2.5.2. คุณสมบัติ ข้อดีและส่วนที่ปรับปรุงจากเอเอสพี (ASP) ของเอเอสพี คอตเน็ต (ASP.NET)

เทคโนโลยี เอเอสพี คอตเน็ต (ASP.NET) ได้มีการปรับปรุงโครงสร้างการทำงานใหม่ จึงทำให้การใช้งานต่างไปจากเอเอสพี (ASP) โดย เอเอสพี คอตเน็ต (ASP.NET) มีลักษณะดังต่อไปนี้

- ในการเขียนสคริปเอเอสพี คอตเน็ต (ASP.NET) ไม่จำเป็นต้องศึกษาภาษาเอชทีเอ็มแอล (HTML) มากนัก ศึกษาเพียงเบื้องต้นว่าแท็ก (tag) คืออะไร มีหน้าตาอย่างไร <html>, <head>, <body>, <form> ใช้ทำอะไร วิธีสร้างฟอร์มเพื่อรับข้อมูลจากผู้ใช้ทำอย่างไรก็เพียงพอแล้ว
- ในการพัฒนาเว็บฟอร์ม (Web Forms) ส่วนติดต่อผู้ใช้แต่ละตัว (User Interface Elements) จะถูกสร้างมาจาก Control ที่สามารถใช้วิธี drag&drop ได้ การที่ใช้คอนโทรล ทำให้ third parties สามารถสร้างคอนโทรล ใหม่ขึ้นมาได้
- ในการเขียนโค้ดส่วนติดต่อผู้ใช้กับโค้ดควบคุมการทำงานจะถูกแยกออกจากกัน (อยู่ในไฟล์เดียวกันหรือคนละไฟล์ก็ได้) สามารถเขียนโค้ดขึ้นมาตอบสนองต่อเหตุการณ์ที่เกิดขึ้นกับคอนโทรลแต่ละตัวได้ ทำให้ง่ายต่อการปรับปรุงหรือหาข้อผิดพลาด
- ในส่วนวิธีติดต่อกับผู้ใช้มีการใช้ เซิร์ฟเวอร์คอนโทรลซึ่งเป็นคอนโทรลที่ถูกประมวลผลบนฝั่งเซิร์ฟเวอร์ ทำให้ควบคุมและเขียนโปรแกรมเพื่อใช้งานได้ง่าย เมื่อต้องการข้อมูลที่ผู้ใช้กรอกเข้ามาทางช่องรับข้อมูล ก็สามารถอ่านข้อมูลมาจากคอนโทรลได้โดยตรง ด้วยการเขียนโค้ดขึ้นมารับค่าพรอพเพอร์ตี้ของคอนโทรลตัวที่เราต้องการ ในทางตรงกันข้ามหากต้องการแสดงข้อมูลกลับไปให้ผู้ใช้ ก็เพียงเขียนโค้ดกำหนดค่าพรอพเพอร์ตี้ให้กับคอนโทรลตัวนั้นๆ
- สามารถนำข้อมูลจากฐานข้อมูลมาแสดงในตารางให้ผู้ใช้ดูได้ โดยไม่ต้องรู้วิธีการสร้างตารางด้วยเอชทีเอ็มแอล (HTML) เลย เพราะมีคอนโทรลมาช่วยทำหน้าที่นี้ให้

- การหาข้อผิดพลาดของ script (Debug) ให้ข้อมูลที่เพียงพอและค่อนข้างละเอียด โดยจะมีคำอธิบายข้อผิดพลาด โค้ดบริเวณที่มีข้อผิดพลาดพร้อมเลขบรรทัด ตลอดจนชื่อไฟล์และหมายเลขบรรทัดที่ผิดอีกครั้งหนึ่ง ทำให้เราเข้าใจข้อผิดพลาดและไปยังจุดผิดได้เร็วกว่าเดิม
- ASP.NET พยายามให้โปรแกรมเมอร์พัฒนาโปรแกรมในลักษณะ event-driven มากกว่าแบบ web scripting โดยเฟรมเวิร์กจะรวมเทคโนโลยีที่มีในปัจจุบัน เช่น JavaScript และคอมโพเนนต์ภายในให้อยู่ในลักษณะของ "view state" คือเปลี่ยนแปลงจาก persistent state เป็น stateless web environment

2.6. Visual Studio .NET

2.6.1. ภาพรวมของวิซวลสตูดิโอ (Visual Studio .NET)

ไมโครซอฟท์ วิซวลสตูดิโอ (Microsoft Visual Studio .NET) คือไอดีอี (integrated development environment) ที่พัฒนาโดยไมโครซอฟท์เป็น general-purpose development tool สำหรับการสร้าง enterprise-grade Microsoft .NET application ให้สามารถเชื่อมต่อกันได้ โดยผู้ที่ออกแบบมีจุดประสงค์สำคัญดังต่อไปนี้

- รองรับ เอ็กซ์เอ็มแอลเว็บเซอร์วิส (XML Web services) คือ แอปพลิเคชันสามารถรับการร้องขอหรือข้อมูล โดยใช้ XML บน HTTP ซึ่ง เอ็กซ์เอ็มแอลเว็บเซอร์วิส (XML Web services) สามารถถูกเข้าถึงได้ด้วยทุกๆ ภาษา และทุกๆ operating system
- โปรแกรมเมอร์สร้างแอปพลิเคชันที่สามารถเข้าถึงข้อมูลได้ทุกที่ , ทุกเวลา และทุกอุปกรณ์ของคอมพิวเตอร์หรือเซิร์ฟเวอร์หลัก
- โปรแกรมเมอร์สามารถสร้างแอปพลิเคชันโดยใช้ประโยชน์ของ platform ได้อย่างเต็มที่ที่มีฟังก์ชันการใช้งาน , ความน่าเชื่อถือ , ประสิทธิภาพและความปลอดภัยสูงสุด
- รองรับ tool และ component จาก third-party vendor ให้โปรแกรมเมอร์สามารถทำงานเสร็จได้อย่างรวดเร็ว

2.6.1.1. ไอดีอี (Integrated Development Environment)

ไอดีอี (Integrated Development Environment (IDE)) คือซอฟต์แวร์ที่ช่วยโปรแกรมเมอร์ในการพัฒนาโปรแกรมประกอบไปด้วย source code editor, compiler หรือ interpreter, build-automation tools , graphical user interface builder และ debugger

สำหรับการพัฒนาโปรแกรมแบบ object oriented แล้วจะมีการรวมเอา class browser, object inspector และ class hierarchy diagram ไว้ด้วย

2.6.1.2. คอมไพเลอร์ (Compiler)

Compiler คือโปรแกรมที่ทำหน้าที่แปลโปรแกรมภาษาต่างๆ ให้กลายเป็นภาษาเครื่อง โดยผลลัพธ์ที่ได้จากการคอมไพล์เรียกว่า object module

Compiler ส่วนใหญ่จะแบ่งออกเป็นสองส่วนคือ front end ซึ่งมีหน้าที่แปล source language ให้เป็น intermediate code ส่วนที่สองคือ back end ที่จะมุ่งไปที่การสร้าง output ที่ถูกต้องและมีประสิทธิภาพ ทำให้มีประโยชน์ในเรื่องสามารถใช้แบ็กเอนด์ (back end) อันเดียวกับหลายๆ source language ได้ และสามารถใช้อย่างหลายๆ back end เพื่อสร้างหลายๆแพลตฟอร์มได้

Compiler front end มีขั้นตอนดังต่อไปนี้

- Lexical analysis
- Syntax analysis
- Semantic analysis
- Intermediate representation

Compile back end มีขั้นตอนดังต่อไปนี้

- Compiler Analysis
- Optimization
- Code generation

Compiler มีสองชนิดคือ native compiler กับ cross-compile

2.6.1.2.1. เนทีฟ คอมไพเลอร์ (Native Compiler)

เนทีฟ คอมไพเลอร์ (Native Compiler) คือ คอมไพเลอร์ ที่สร้างโค้ดเพื่อที่จะรันบนคอมพิวเตอร์และ OS (หรือ platform) เดียวกันกับที่คอมไพเลอร์รันอยู่

2.6.1.2.2. ครอส คอมไพเลอร์ (Cross Compiler)

ครอส คอมไพเลอร์ (Cross Compiler) คือคอมไพเลอร์ ที่สร้างโค้ดเพื่อที่จะรันบนแพลตฟอร์มอื่นที่ต่างออกไป โดยจะมีประโยชน์มากเมื่อมีการนำฮาร์ดแวร์ แพลตฟอร์มใหม่มาในครั้งแรก

2.6.1.3. Debugger

Debugger เป็นโปรแกรมที่ใช้สำหรับค้นหาข้อผิดพลาดในการทำงานจาก source code ที่ถูกเขียนขึ้นมา เมื่อโปรแกรมเกิดการชนหรือมีบั๊ก debugger จะแสดงตำแหน่งของโค้ดนั้นๆ ขึ้นมา (หากเป็นแบบ source-level debugger หรือ symbolic debugger)

โดยปกติแล้ว Debugger จะมีฟังก์ชันที่ซับซ้อน เช่น สามารถรันโปรแกรมแบบ step by step (single-stepping), stopping (breaking) คือหยุดโปรแกรมเพื่อตรวจสอบสถานะปัจจุบัน และติดตามค่าของตัวแปร เป็นต้น

2.6.2. คุณสมบัติใหม่ของ Visual Studio .NET 2003

คุณสมบัติใหม่และคุณสมบัติสำคัญของ Visual Studio .NET 2003 มีดังต่อไปนี้

- สนับสนุน คอเทรเน็ทเฟรมเวิร์ก (.NET Framework) 1.1 อย่างเต็มรูปแบบ
- สนับสนุนมาตรฐานล่าสุดของ เว็บเซอร์วิส (Web services)

รวมถึงการกำหนดเส้นทางสื่อสาร, ความปลอดภัย, และการแนบข้อมูลให้กับ

Web Services Enhancements

- สนับสนุนอุปกรณ์มือถือและ Embedded Device

Visual Studio .NET 2003 สนับสนุน .NET Compact Framework อย่างเต็มรูปแบบ ทำให้ อุปกรณ์มือถือ เช่น Pocket PC ก้าวไปสู่การใช้เทคโนโลยี .NET ย่างขึ้น ต่างจากเดิมที่ต้องการ tool kit แยกต่างหาก

- ช่วยให้การนำแอปพลิเคชัน ไปใช้งานและบำรุงรักษาง่ายขึ้น

Visual Studio (Visual Studio .NET) 2003 จะแก้ปัญหาส่วนใหญ่ที่เกี่ยวข้องกับการนำ แอปพลิเคชัน ไปใช้ และจัดการปัญหาความขัดแย้งของเวอร์ชันของ DLL

- ปรับปรุงการทำงานของ Visual Basic .Net Upgrade Wizard

ด้วยเครื่องมือและคำแนะนำที่เตรียมไว้ จะช่วยให้คุณอัพเกรดชุดคำสั่งและ แอปพลิเคชันจาก Visual Basic 6.0 ได้อย่างง่ายดาย

- สภาพแวดล้อมในการพัฒนาที่เป็นหนึ่งเดียว

Visual Studio (Visual Studio .NET) 2003 รวมสภาพแวดล้อมในการทำงานเข้ากับ เครื่องมือที่จำเป็นสำหรับการพัฒนา เพื่อรองรับกับแอปพลิเคชันทุกรูปแบบ เครื่องมือออกแบบที่ ใช้งานร่วมกันยังทำให้พัฒนา application บน Windows และอุปกรณ์มือถือได้ในรูปแบบเดียวกัน อีกด้วย ผลที่ได้ก็คือ นักพัฒนาสามารถนำทักษะมาใช้กับงานได้หลายรูปแบบและเลือกใช้ภาษา โปรแกรมได้มากมาย

ด้วยการสนับสนุนภาษาโปรแกรมต่างๆ มากมาย จึงสามารถเขียนโปรแกรมด้วยภาษาที่ ต้องการ และรวมเข้ากับโปรแกรมที่เขียนขึ้นด้วยภาษาใดก็ได้ นักพัฒนาจึงสามารถใช้ความ ชำนาญที่มีอยู่ในปัจจุบันทำงานต่อไปโดยไม่ต้องฝึกอบรมใหม่

- HTML Designer

พัฒนา HTML, ASP.NET Web Forms โดยไม่ต้องเขียนชุดคำสั่ง HTML หรือสคริปต์

Microsoft Windows Forms , Web Forms และ Mobile Web Forms

- การพัฒนาสำหรับ server

Server Explorer และเครื่องมืออื่นๆ ได้รวมองค์ประกอบของ application ที่อยู่บน server เพื่อเรียกใช้เครื่องมือสำหรับการจัดการ server, event logs, database และ Web services ได้อย่างสะดวก

- Visual Studio .NET Debugger

ใช้งาน remote debugger ได้

- Dynamic Help และ Task List

จะช่วยให้จัดการขั้นตอนการพัฒนาได้อย่างเป็นระบบ โดยใส่หมายเหตุประกอบชุดคำสั่ง, การตรวจสอบและเข้าถึงข้อผิดพลาดและเตือนได้อย่างรวดเร็ว

- Visual Basic .NET

สนับสนุนการเขียนโปรแกรมเชิงวัตถุ (OOP) อย่างเต็มรูปแบบ รวมถึงคุณสมบัติการถ่ายทอด (Inheritance), การจัดการ exception แบบมีโครงสร้างและ free-threading

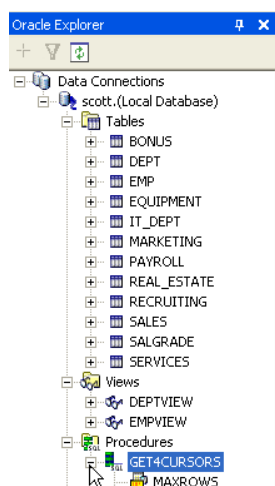
2.7. Oracle Developer Tools for Visual Studio .NET

2.7.1. ส่วนประกอบต่างๆ ของ Oracle Developer Tools Visual Studio .NET

เป็นเครื่องมือแอดอิน (add-in) ของออราเคิลที่ช่วยให้นักพัฒนาที่ใช้ วิวอลสตูดิโอโคดอทเน็ต (Visual Studio .NET) สามารถพัฒนาโปรแกรมได้รวดเร็วยิ่งขึ้น โดยมีส่วนประกอบต่างๆ ดังนี้

2.7.1.1. Oracle Explorer

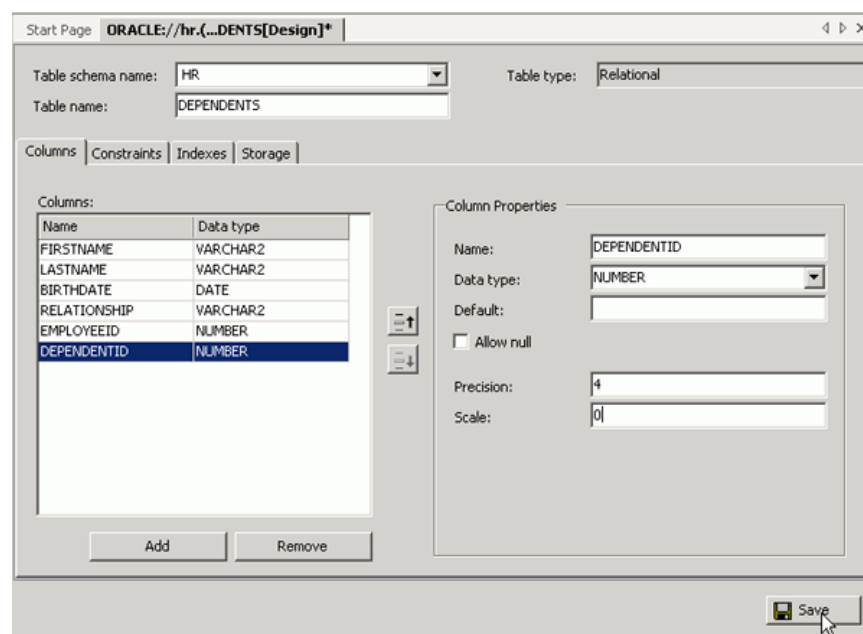
เป็นหน้าต่าง tree control ที่แสดงให้เห็นโครงสร้างของ Oracle Schema เช่น tables , stored procedures เมื่อ schema object ถูกเลือกจะมีการแสดง metadata ไว้ที่ “properties pane” ดังรูป 2.24.



รูปที่ 2.24. แสดง Oracle Explorer

2.7.1.2. Designers and Wizards

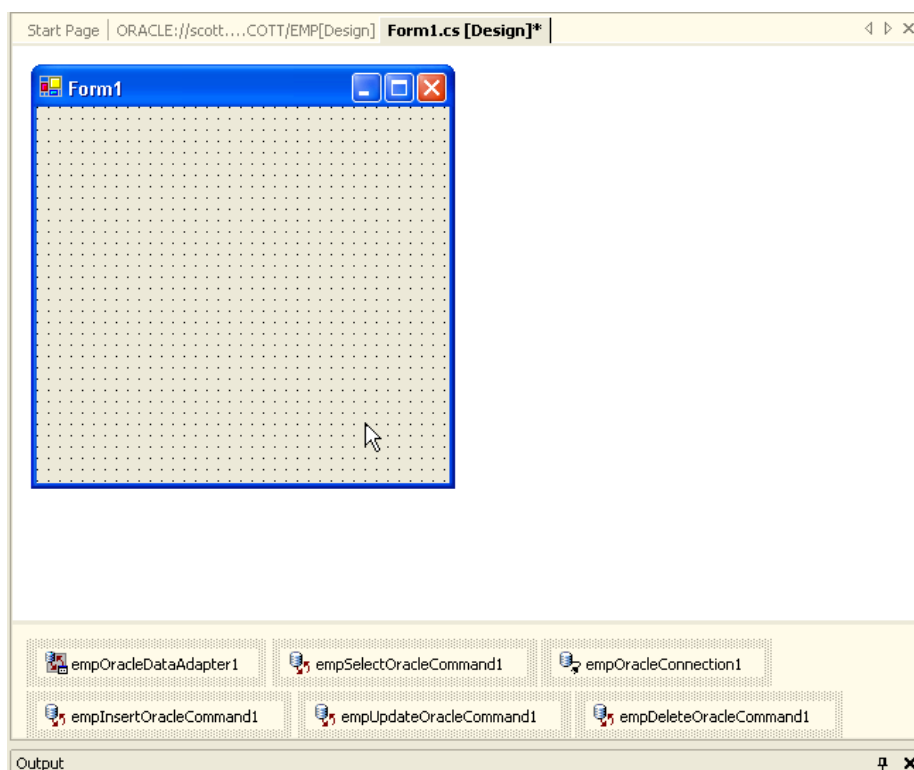
เป็นเครื่องมือ (wizard) ที่ทำให้การสร้างหรือเปลี่ยนแปลง database objects เช่น tables, views, stored procedures, stored functions, PL/SQL packages, sequences, indexes, constraints, triggers, synonyms และอื่นๆ ทำได้ง่ายขึ้นในรูปแบบ GUI ไม่จำเป็นต้องหาจาก datatype หรือจำ SQL syntax สามารถเลือก column ต่างๆ ได้ง่ายๆ จาก table name และ name of column เลือก datatype ได้จาก drop down box เมื่อกด "save" จะปรากฏ schema object ที่ Oracle explorer ดังรูป 2.25.



รูปที่ 2.25. แสดง Designers and Wizards

2.7.1.3. Automatic Code Generation

เมื่อทำการ drag&drop ตัว schema object จาก Oracle Explorer ไปบน application's form มันจะทำการสร้างโค้ดในส่วนของการ SELECT , UPDATE , INSERT, และ DELETE ของโครงสร้าง object นั้นให้ โดยมีคลาส OracleDataAdapter เป็นตัวเชื่อมต่อ User Interface element (เช่น datagrid) บน application's form ไปยัง Oracle database ด้วยการเพิ่มโค้ดเพียงเล็กน้อย โดยมีลักษณะ ดังรูป 2.26.



รูปที่ 2.26 แสดง Automatic Code Generation

2.7.1.4. PL/SQL Editor

มีการรวม development environment สำหรับ Oracle developers ที่ต้องการพัฒนา stored procedure หน้าต่าง PL/SQL Editor มีการใช้สีและ auto completion เพื่อความสะดวก รวดเร็วในการพัฒนา application หากทำการ compile แล้วเกิด error จะมีข้อมูลนั้นอยู่ใน error task list และสามารถคลิกเพื่อที่จะไปยังบรรทัดที่เกิด error ได้ ดังรูป 2.27.

```

PROCEDURE "SCOTT"."GET4CURSORS" ("MAXROWS" IN NUMBER, "EMPCUR" OUT SYS_REFCURSOR,
SYS_REFCURSOR, SALGRADECUR OUT SYS_REFCURSOR, BONUSCUR OUT SYS_REFCURSOR)

BEGIN
    OPEN DEPTCUR FOR SELECT * FROM DEPT;
    OPEN SALGRADECUR FOR SELECT * FROM SALGRADE;
    OPEN BONUSCUR FOR SELECT * FROM BONUS;

END "GET4CURSORS";
  
```

รูปที่ 2.27. แสดง PL/SQL Editor

2.7.1.5. Stored Procedure Testing

สามารถทดสอบ stored procedures และ function ได้อย่างรวดเร็ว โดยการกด "Run" ที่ Oracle Explorer (โดยใส่ parameters ที่ต้องการไปด้วย) output ที่ออกมาอยู่ในรูปแบบที่อ่านง่าย สำหรับ output ที่ซับซ้อนเมื่อคลิกที่ลิงค์แล้ว จะเปิดตารางที่ประกอบด้วยข้อมูลที่ซับซ้อนออกมาให้สามารถอ่านได้ง่าย ดังรูป 2.28.

Procedure <SCOTT.GET4CURSORS@scott.(Local Database)> was run successfully.

Parameters:

Name	Direction	Data Type	Value
MAXROWS	IN	NUMBER	1000
EMPCUR	OUT	REF_CURSOR	<Click here for details...>
DEPTCUR	OUT	REF_CURSOR	<Click here for details...>
SALGRADECUR	OUT	REF_CURSOR	<Click here for details...>
BONUSCUR	OUT	REF_CURSOR	<Click here for details...>

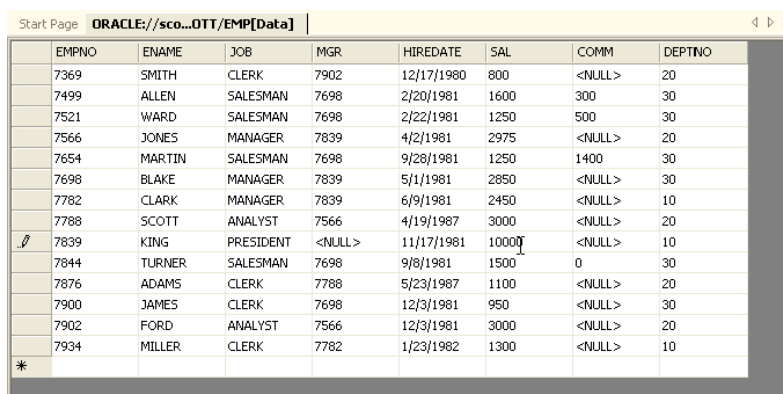
Parameter Details - EMPCUR:

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7369	SMITH	CLERK	7902	12/17/1980	800	(null)
7499	ALLEN	SALESMAN	7698	2/20/1981	1600	300
7521	WARD	SALESMAN	7698	2/22/1981	1250	500
7566	JONES	MANAGER	7839	4/2/1981	2975	(null)
7654	MARTIN	SALESMAN	7698	9/28/1981	1250	1400
7698	BLAKE	MANAGER	7839	5/1/1981	2850	(null)
7782	CLARK	MANAGER	7839	6/9/1981	2450	(null)
7788	SCOTT	ANALYST	7566	4/19/1987	3000	(null)
7839	KING	PRESIDENT	(null)	11/17/1981	10000	(null)
7844	TURNER	SALESMAN	7698	9/8/1981	1500	0
7876	ADAMS	CLERK	7788	5/23/1987	1100	(null)
7900	JAMES	CLERK	7698	12/3/1981	950	(null)

รูปที่ 2.28. แสดง Stored Procedure Testing

2.7.1.6. Oracle Data Windows

แสดง table หรือ view data ที่ง่ายต่อการอ่าน และอนุญาตให้ผู้ใช้สามารถ INSERT, UPDATE , DELETE ข้อมูลได้โดยไม่ต้องมี Visual Studio .NET development environment ถ้าตารางมีขนาดใหญ่ ก็สามารถที่จะเลือกข้ามไปที่ row นั้นๆ ได้ ดังรูป 2.29.



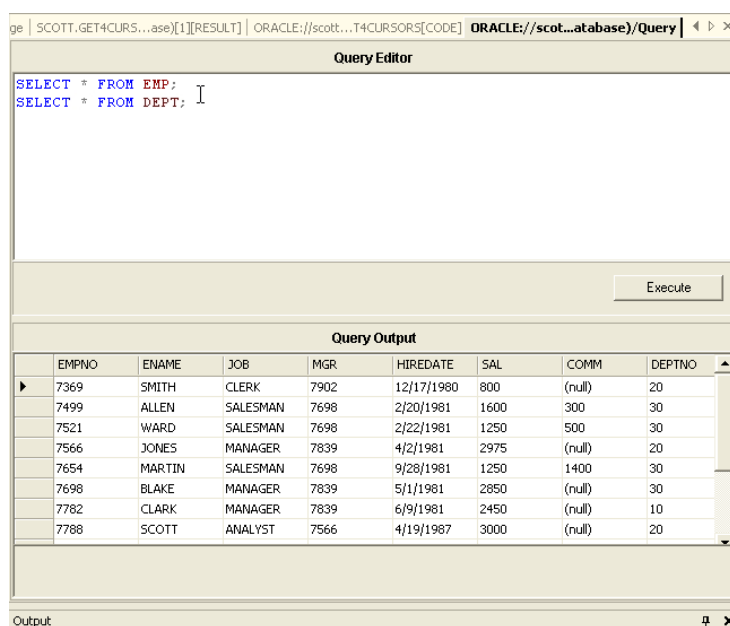
The screenshot shows the Oracle Data Windows interface with the title bar 'ORACLE://sco...OTT/EMP[Data]'. The window displays a table with the following columns: EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, and DEPTNO. The data is as follows:

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	12/17/1980	800	<NULL>	20
7499	ALLEN	SALESMAN	7698	2/20/1981	1600	300	30
7521	WARD	SALESMAN	7698	2/22/1981	1250	500	30
7566	JONES	MANAGER	7839	4/2/1981	2975	<NULL>	20
7654	MARTIN	SALESMAN	7698	9/28/1981	1250	1400	30
7698	BLAKE	MANAGER	7839	5/1/1981	2850	<NULL>	30
7782	CLARK	MANAGER	7839	6/9/1981	2450	<NULL>	10
7788	SCOTT	ANALYST	7566	4/19/1987	3000	<NULL>	20
7839	KING	PRESIDENT	<NULL>	11/17/1981	10000	<NULL>	10
7844	TURNER	SALESMAN	7698	9/8/1981	1500	0	30
7876	ADAMS	CLERK	7788	5/23/1987	1100	<NULL>	20
7900	JAMES	CLERK	7698	12/3/1981	950	<NULL>	30
7902	FORD	ANALYST	7566	12/3/1981	3000	<NULL>	20
7934	MILLER	CLERK	7782	1/23/1982	1300	<NULL>	10

รูปที่ 2.29. แสดง Oracle Data Windows

2.7.1.7. SQL Query Windows

ใช้สำหรับ Execute ad-hoc SQL statement หรือ script เพื่อความสะดวกในการตรวจสอบความถูกต้องของผลลัพธ์ที่ถูก query ออกมา สามารถใช้คำสั่งแบบ multiple statement ได้ (output ใหม่จะต่อท้าย output เก่า) ดังรูป 2.30.



The screenshot shows the SQL Query Windows interface. The title bar indicates the current query is 'SCOTT.GET4CURS...ase)(1)[RESULT]'. The 'Query Editor' contains the following SQL statements:

```
SELECT * FROM EMP;
SELECT * FROM DEPT;
```

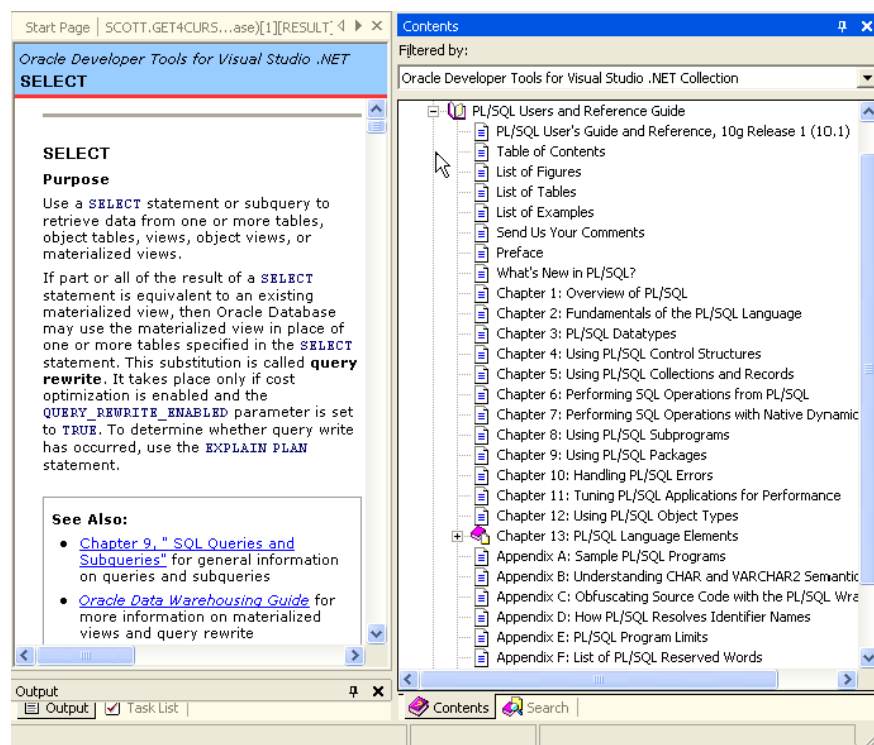
The 'Execute' button is visible. Below the editor, the 'Query Output' section displays the results of the first query (SELECT * FROM EMP):

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	12/17/1980	800	(null)	20
7499	ALLEN	SALESMAN	7698	2/20/1981	1600	300	30
7521	WARD	SALESMAN	7698	2/22/1981	1250	500	30
7566	JONES	MANAGER	7839	4/2/1981	2975	(null)	20
7654	MARTIN	SALESMAN	7698	9/28/1981	1250	1400	30
7698	BLAKE	MANAGER	7839	5/1/1981	2850	(null)	30
7782	CLARK	MANAGER	7839	6/9/1981	2450	(null)	10
7788	SCOTT	ANALYST	7566	4/19/1987	3000	(null)	20

รูปที่ 2.30. แสดง SQL Query Windows

2.7.1.8. Integrated Help System

มีการรวบรวมเอกสารสำคัญของ Oracle ให้อยู่ในรูปแบบของ Visual Studio help format เช่น The SQL Language Reference Guide, the PL/SQL User and Reference Guide, และ the Error Messages and Codes และรวมความสามารถของ Visual Studio context sensitive help ไว้ด้วย ดังรูป 2.31.



รูปที่ 2.31. แสดง Integrated Help System

2.8. โครงสร้างข้อมูลแบบกราฟ (Graph)

กราฟ(Graph) เป็นโครงสร้างข้อมูลที่ใช้แสดงความสัมพันธ์ระหว่างออบเจกต์ (object) โดยจะประกอบไปด้วยกลุ่มของโหนด (vertex) ที่ใช้แทนออบเจกต์และกลุ่มของเส้นเชื่อม (Edge) ระหว่างโหนดกรณีที่ออบเจกต์ตั้งแต่ 2 ออบเจกต์ขึ้นไปมีความสัมพันธ์กัน ก็จะมีเส้นเชื่อมระหว่างออบเจกต์หรือโหนดเหล่านั้น

โครงสร้างของกราฟนั้นจะประกอบไปด้วยโหนดและเส้นเชื่อมระหว่างโหนด (กรณีที่โหนดแต่ละโหนดนั้นมีความสัมพันธ์กัน) โดยสามารถเขียนแทนด้วยสัญลักษณ์ต่อไปนี้

$G = (V, E)$ เมื่อ G คือกราฟ

V คือกลุ่มของโหนด

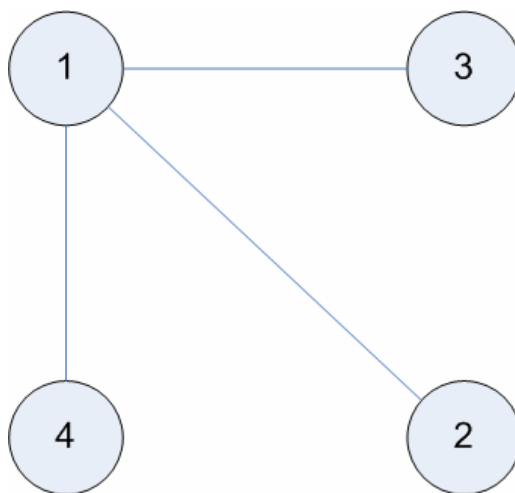
E คือเส้นเชื่อมระหว่างโหนด

2.8.1. ประเภทของกราฟ

ในส่วน of ประเภทของกราฟนั้นจะแบ่งออกได้เป็น 3 ประเภทหลักๆ คือ Direct graph , Undirect graph และ Cycle graph

2.8.1.1. Undirect graph

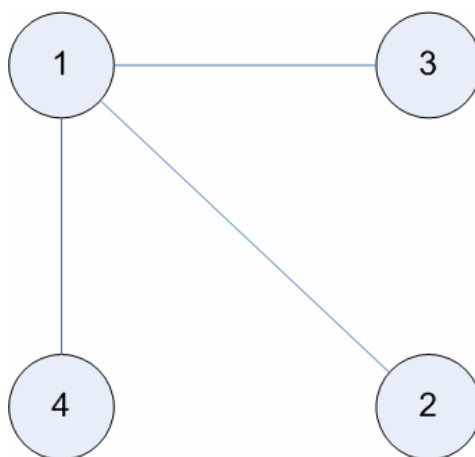
Undirect graph นี้มีลักษณะเป็นกราฟที่มีเส้นที่เชื่อมต่อระหว่างโหนดในแต่ละโหนด โดยเส้นที่เชื่อมต่อนี้จะไม่มีการแสดงทิศทางของการเชื่อมต่อ ดังรูป 2.32.



รูปที่ 2.32. แสดงตัวอย่างของ Undirect graph

2.8.1.2. Direct graph

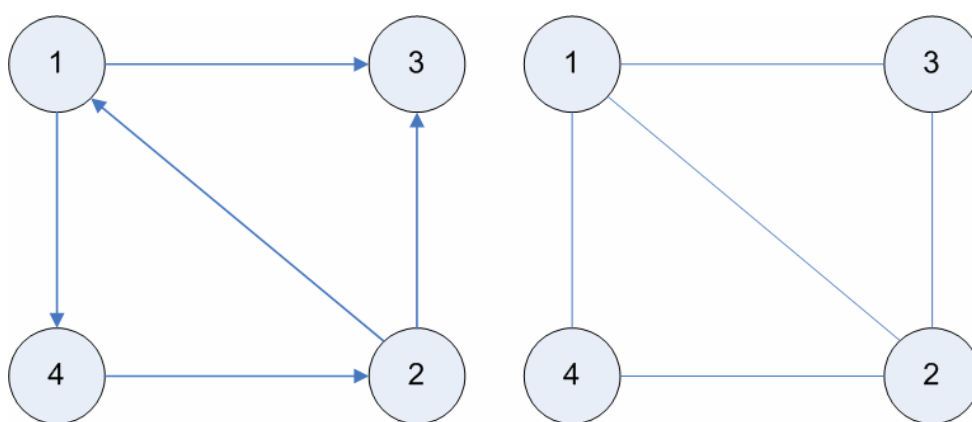
Direct graph หรืออีกชื่อหนึ่งคือ Digraph จะมีลักษณะเป็นกราฟที่มีเส้นเชื่อมต่อระหว่างโหนดในแต่ละโหนด โดยเส้นเชื่อมของ Direct graph นี้จะไม่เหมือนกับเส้นเชื่อมของ Undirect graph ตรงที่เส้นเชื่อมของ Direct graph นั้นจะมีการแสดงทิศทางของเส้นเชื่อมต่อดังรูป 2.33.



รูปที่ 2.33. แสดงตัวอย่างของ Direct graph

2.8.1.3. Cyclic graph

Cyclic graph นี้จะเป็นกราฟที่เส้นทาง (path) เกิดจากเส้นเชื่อมระหว่างโหนดที่มีลักษณะเป็นวงจรปิด (Cycle) คือหมายถึงมีโหนดต้นทางและโหนดปลายทางเป็นโหนดเดียวกัน โดย Cyclic graph สามารถเป็นได้ทั้ง Direct graph และ Undirect graph ดังรูป 2.34.



รูปที่ 2.34 แสดงตัวอย่างของ Cyclic graph

2.9. การค้นหาข้อมูล (searching)

การค้นหาคำตอบ หรือการค้นหาข้อมูลในทางคอมพิวเตอร์มักจะกระทำบนโครงสร้างข้อมูลแบบต้นไม้ และกราฟ ทั้งนี้เพราะโครงสร้างข้อมูลในลักษณะนี้สามารถทำให้การค้นหาทำได้สะดวกและสามารถพลิกแพลงการค้นหาได้ง่าย ดังนั้นในการค้นหาข้อมูลที่มีขนาดใหญ่ขึ้น ก่อนการค้นหา หรือระหว่างการค้นหา ข้อมูลที่จะถูกค้นจะต้องถูกจัดให้อยู่ในรูปแบบของต้นไม้ หรือกราฟเท่านั้น การค้นหาข้อมูลบนโครงสร้างต้นไม้และกราฟสามารถจำแนกได้ 2 แบบคือ การค้นหาแบบโบลด์(Blind Search) และการค้นหาแบบฮิวริสติก(Heuristic Search)

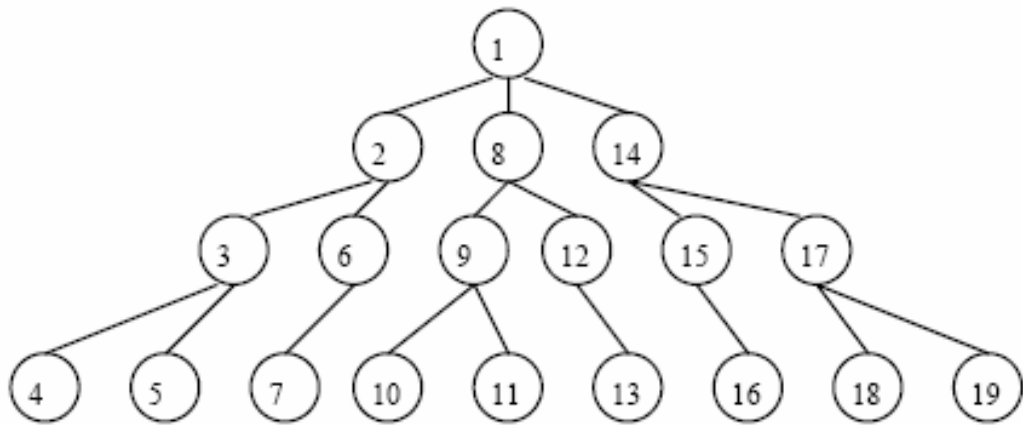
2.9.1. การค้นหาแบบโบลด์ (Blind Search)

การค้นหาแบบโบลด์(Blind search) เป็นการค้นหาแบบที่เดินทางจากโหนดหนึ่งไปยังอีกโหนดหนึ่ง โดยอาศัยทิศทางเป็นตัวกำหนดการค้นหา และไม่ต้องมีข้อมูลอะไรมาช่วยเสริมในการตัดสินใจว่าจะเดินทางต่อไปอย่างไร หรือกล่าวอย่างง่าย ๆ คือการจะหยิบข้อมูลใดมาช่วยในการค้นหาต่อไปนั้นจะไม่ต้องอาศัยข้อมูลใด ๆ ทั้งสิ้น นอกจากทิศทางซึ่งเป็นรูปแบบตายตัว ซึ่งการค้นหาแบบโบลด์สามารถแบ่งย่อยได้ดังนี้ คือ การค้นหาทั้งหมด และการค้นหาบางส่วน

- การค้นหาทั้งหมด(exhaustive search) คือ การค้นหาทั้งหมดของปริภูมิสถานะ
- การค้นหาบางส่วน (partial search) การค้นหาเพียงบางส่วนของปริภูมิสถานะ ซึ่งในความเป็นจริงการค้นหาส่วนมากใช้การค้นหาเฉพาะบางส่วนเท่านั้นเนื่องจากปริภูมิสถานะมักมีขนาดใหญ่ ทำให้ไม่สามารถค้นหาได้ทั้งหมด ดังนั้นจึงมีความเป็นไปได้ว่าคำตอบที่ได้ อาจไม่ใช่คำตอบที่ดีที่สุด ซึ่งการค้นหาแบบนี้สามารถแบ่งได้เป็น 2 ประเภทคือ การค้นหาแบบลึกก่อน(Depth first search) และการค้นหาแบบกว้างก่อน (Breadth first search)

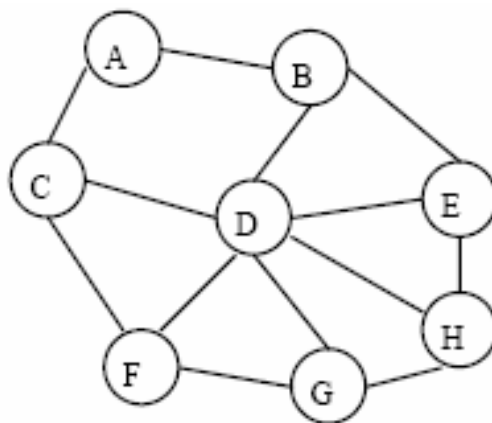
2.9.1.1. การค้นหาแบบลึกก่อน(Depth first search)

การค้นหาแบบลึกก่อนเป็นการค้นหาที่กำหนดทิศทางจากรูปของโครงสร้างต้นไม้ ที่เริ่มต้นจากโหนดราก(Root node) ที่อยู่บนสุด แล้วเดินทางมาให้ลึกที่สุด เมื่อถึงโหนดล่างสุด (Terminal node) ให้ย้อนขึ้นไปที่จุดสูงสุดของกิ่งเดียวกันที่มีกิ่งแยกและยังไม่ได้เดินทาง แล้วเริ่มเดินทางจนถึงโหนดลึกสุดอีก ทำเช่นนี้สลับไปเรื่อยจนพบโหนดที่ต้องการหาหรือสำรวจครบทุกโหนดแล้วตามรูปที่ 2.35 การค้นหาแบบลึกก่อนจะมีลำดับการเดินทางตามโหนดดังตัวเลขที่กำกับไว้ในแต่ละโหนด



รูปที่ 2.35. ลำดับการเดินทางบนโหนดของการค้นหาแบบลึกก่อนบนโครงสร้างต้นไม้

โครงสร้างข้อมูลที่ใช้สำหรับการค้นหาซึ่งจะสามารถใช้กับโครงสร้างกราฟได้ด้วย โดยอาศัยหลักการเดียวกัน แต่สำหรับการเดินทางบนกราฟนั้นจะไม่มีโหนดลึกสุดดังนั้นการเดินทางบนกราฟจะต้องปรับวิธีการเป็นดังนี้ โดยเริ่มจากโหนดเริ่มต้น จากนั้นให้นำโหนดที่อยู่ติดกับโหนดที่กำลังสำรวจอยู่ (ที่ยังไม่ได้ทำการสำรวจและยังไม่ได้อยู่ในแสต็กมาใส่แสต็ก) มาเก็บไว้ในแสต็กเมื่อสำรวจโหนดนั้นเสร็จ ให้พอป (pop) ตัวบนสุดของโหนดออกมาทำการสำรวจ แล้วนำโหนดข้างเคียงทั้งหมดที่ยังไม่ได้สำรวจมาต่อท้ายแสต็ก แล้วพอปตัวบนสุดออกมาสำรวจ ทำเช่นนี้เรื่อย ๆ จนกระทั่งพบโหนดที่ต้องการ หรือสำรวจครบทุกโหนด ดังรูปที่ 2.36.



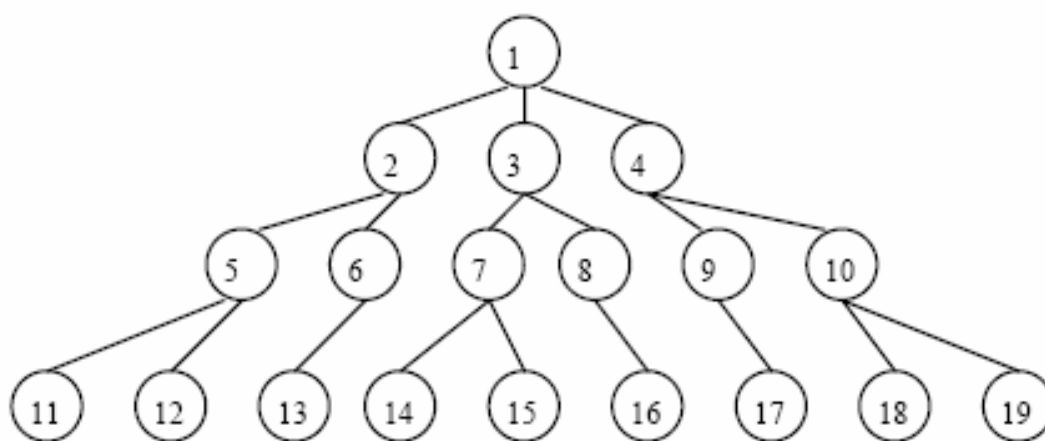
รูปที่ 2.36. โครงสร้างข้อมูลแบบกราฟ

การสำรวจจะเริ่มต้นที่ A และนำโหนดข้างเคียง B และ C มาเก็บไว้ในแสต็ก เมื่อสำรวจ A เสร็จพอปข้อมูลจากแสต็กออกมาได้ C ทำการสำรวจ C และนำโหนดข้างเคียงกับ C ที่ยังไม่ได้ทำการสำรวจและยังไม่ได้อยู่ในแสต็กมาใส่แสต็ก D และ F พุช(Push) ใส่แสต็ก ดังนั้นในแสต็กตอนนี้มี B D F อยู่ เมื่อสำรวจ C เสร็จ พอป F ออกมาทำการสำรวจ แล้วนำโหนดข้างเคียงที่ยัง

ไม่ได้สำรวจและยังไม่ได้อยู่ในแอสต์มาใส่แอสต์ ซึ่งก็คือ G ดังนั้นข้อมูลในแอสต์จะเป็น B D G ทำเช่นนี้ไปเรื่อย ๆ จนจบการทำงานก็จะได้ลำดับการสำรวจคือ (A C F G H E D B)

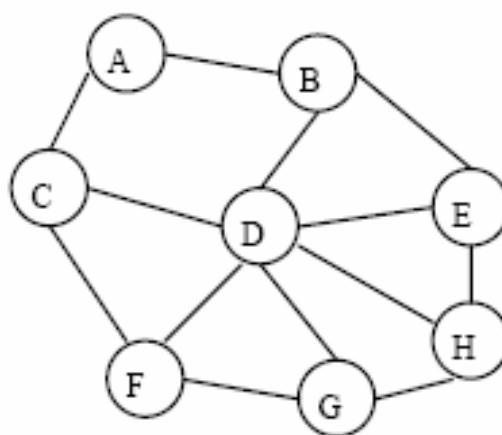
2.9.1.2. การค้นหาแบบกว้างก่อน (Breadth first search)

การค้นหาแบบกว้างก่อนนี้จะเป็นการกำหนดทิศทางการค้นหาแบบที่แต่ละระดับของโครงสร้างต้นไม้โดยเริ่มจากโหนดราก(ระดับที่ 0) แล้วลงมาระดับที่ 1 จากซ้ายไปขวา เมื่อเสร็จระดับที่ 1 ไประดับที่ 2 จากซ้ายไปขวาเช่นกัน ทำเช่นนี้เรื่อย ๆ จนพบโหนดที่ต้องการ ดังรูปที่ 2.37.



รูปที่ 2.37. ลำดับการค้นหาแบบกว้างก่อนบนโครงสร้างต้นไม้

สำหรับการค้นหาแบบกว้างก่อนบนโครงสร้างต้นไม้ จะอาศัยโครงสร้างข้อมูลแบบคิว (Queue) มาช่วย และด้วยวิธีการเช่นเดียวกับการค้นหาแบบลึกก่อนคือ ให้เริ่มต้นสำรวจที่โหนดเริ่มต้น แล้วนำโหนดข้างเคียงเก็บไว้ในคิว เมื่อสำรวจโหนดเริ่มต้นเสร็จให้นำข้อมูลในคิวออกมาสำรวจ แล้วนำโหนดข้างเคียงที่ยังไม่ได้สำรวจและไม่ได้อยู่ในคิวใส่คิวไว้ ทำเช่นนี้ไปเรื่อย ๆ จนพบโหนดที่ต้องการ หรือเมื่อสำรวจครบทุกโหนด ดังรูปที่ 2.38.



รูปที่ 2.38. โครงสร้างข้อมูลแบบกราฟ

การสำรวจเริ่มต้นที่ A นำโหนดข้างเคียง B C ไว้ในคิว เมื่อสำรวจ A เสร็จ นำข้อมูลในคิวคือ B ออกมาสำรวจ แล้วนำข้อมูลข้างเคียงคือ D E ใส่คิว ตอนนี้คิวจะมี B D E อยู่ แล้วนำ B ออกมาสำรวจทำเช่นนี้เรื่อย ๆ จะได้ลำดับการสำรวจข้อมูลคือ (A B C D E F G H)

การค้นหาแนวลึกก่อนและการค้นหาแบบแนวกว้างก่อนหน้านี้ มีข้อที่แตกต่างกันหลายประการดังตารางที่ 2.1.

ตารางที่ 2.1. แสดงการเปรียบเทียบการค้นหาระหว่างแบบการค้นหาแนวลึกก่อนกับการค้นหาแนวกว้างก่อน

การค้นหาแนวลึกก่อน (Depth first search)	การค้นหาแนวกว้างก่อน (Breadth first search)
1. ใช้หน่วยความจำน้อยกว่า เพราะว่าสถานะในเส้นทางค้นหาปัจจุบันเท่านั้นที่ถูกเก็บ(ในขณะที่ใดๆ จะเก็บเส้นทางเดียว พอจะไปเส้นทางอื่นเส้นทางที่ผ่านมาก็ไม่จำเป็นต้องเก็บ)	1. ใช้หน่วยความจำมาก เพราะต้องเก็บสถานะไว้ทุกตัวเพื่อหาเส้นทางจากสถานะเริ่มต้นไปหาคำตอบ
2. อาจจะคิดเส้นทางที่ลึกมากโดยไม่พบคำตอบ เช่นในกรณีที่เป็นเส้นทางนั้นไม่มีคำตอบและเป็นเส้นทางที่ยาวไม่สิ้นสุด จะทำให้ไม่สามารถไปเส้นทางอื่นได้	2. จะไม่คิดเส้นทางที่ลึกมาก ๆ โดยไม่พบคำตอบ
3. ถ้าคำตอบอยู่ในระดับ $n+1$ สถานะอื่นทุกตัวที่ระดับ 1 ถึงระดับ n ไม่จำเป็นต้องถูกกระจายจนหมด	3. ถ้าคำตอบอยู่ในระดับ $n+1$ สถานะทุกตัวที่ระดับ 1 ถึงระดับ n จะต้องถูกกระจายจนหมด ทำให้มีสถานะที่ไม่จำเป็นในเส้นทางที่จะไปสู่คำตอบถูกกระจายออกด้วย
4. เมื่อพบคำตอบไม่สามารถรับประกันได้ว่าเส้นที่ได้เป็นเส้นทางที่สั้นที่สุดหรือไม่	4. ถ้ามีคำตอบจะรับประกันได้ว่าจะพบคำตอบแน่ ๆ และจะได้เส้นทางที่สั้นที่สุดด้วย

2.9.2. การค้นหาแบบฮิวริสติก(Heuristic Search)

การค้นหาคำตอบอาศัยวิธีการทางฮิวริสติก (heuristic search) มีความแตกต่างจากการค้นหาข้อมูลแบบธรรมดา ตรงที่การค้นหาข้อมูลธรรมดา ผู้ที่ทำการค้นข้อมูลจะต้องตรวจสอบข้อมูลที่ละตัวทุกตัวจนครบ แต่ฮิวริสติกจะไม่ลงไปดู ข้อมูลทุกตัว วิธีการนี้จะเลือกได้คำตอบที่เหมาะสมให้กับการค้นหา ซึ่งมีข้อดีคือ สามารถทำการ ค้นหาคำตอบจาก ข้อมูลที่มีขนาดใหญ่มา ๆ ได้ แต่มีข้อเสียคือคำตอบที่ได้เป็นเพียงคำตอบที่ดี เท่านั้น ไม่แน่ว่าจะดีที่สุด แต่เนื่องจากว่าปัญหาในบางลักษณะนั้นใหญ่มาก และเป็นไปไม่ได้ที่จะทำ การค้นหาด้วยวิธีธรรมดากระบวนการของฮิวริสติกจึงเป็นสิ่งที่จำเป็นในเรื่องของฮิวริสติกนั้น นอกจากจะมีการค้นหาแบบฮิวริสติกแล้ว ยังมีอีกสิ่งหนึ่งที่สำคัญคือ ฮิวริสติกฟังก์ชัน (heuristic function) ซึ่งหมายถึงฟังก์ชันที่ทำหน้าที่ในการวัดขนาดของความเป็นไปได้ในการแก้ปัญหาซึ่งจะแสดงด้วยตัวเลข วิธีการดังกล่าวจะกระทำ ได้โดยการพิจารณาถึงวิธีการ (aspects) ต่าง ๆ ที่ใช้ในการแก้ปัญหา ณ สถานะหนึ่งว่าจะสามารถแก้ปัญหาได้ตามที่ต้องการหรือไม่ โดยกำหนดเป็นน้ำหนักที่ให้การแก้ปัญหาของแต่ละวิธี น้ำหนักเหล่านี้จะถูกแสดงด้วยตัวเลขที่กำกับไว้กับโหนดต่าง ๆ ในกระบวนการ ค้นหา และค่าเหล่านี้จะเป็นตัวที่ใช้ในการประมาณความเป็นไปได้ว่าเส้นทางที่ผ่านโหนดนั้นจะมี ความเป็นไปได้ในการนำไปสู่หนทางการแก้ปัญหาได้มากน้อยแค่ไหน จุดประสงค์ที่แท้จริงของฮิวริสติก ฟังก์ชันก็คือ การกำกับทิศทางของกระบวนการค้นหา เพื่อให้อยู่ในทิศทางที่ได้ประโยชน์สูงสุด โดยการบอกว่าเราควรเลือกเดินเส้นทางไหนก่อน ในกรณีที่มีเส้นทางมากกว่าหนึ่งเส้นทางต้องเลือกกระบวนการค้นหาแบบฮิวริสติก โดยปกติแล้วจะต้องอาศัยฮิวริสติกฟังก์ชัน ทำให้การแก้ปัญหาหนึ่ง ๆ จะดีหรือไม่ ก็ขึ้นอยู่กับฮิวริสติกฟังก์ชัน ดังนั้นการค้นหาแบบนี้จึงไม่มีอะไรเป็นหลักประกันว่าจะได้สิ่งที่ไม่ดีออกมาด้วยเหตุนี้เอง เราจึงเรียกการ ค้นหาแบบฮิวริสติกนี้ว่า Weak Methods หรือจะกล่าวอีกนัยหนึ่งคือ Weak Methods เป็นกระบวนการควบคุมโดยทั่วไป (general-purpose control strategies)

2.10. การบำรุงรักษาซอฟต์แวร์ (software Maintenance)

2.10.1. ความหมายและภาพรวม

IEEE Computer Society ได้ให้ความหมายคำว่า software maintenance ไว้ว่า คือการที่จะทำให้ ระบบของ software นั้นสามารถทำการ ตรวจสอบความถูกต้อง, พัฒนาประสิทธิภาพ, หรือปรับเปลี่ยนสภาพแวดล้อม ได้ หรือให้ ระบบนั้นสามารถที่จะคงสภาพที่ให้อำนาจการทำงานได้อย่างสภาพปกติ

2.10.2. Level of maintenance

Level 1 Full maintenance: จะมีการ ปรับปรุง เปลี่ยนแปลงระบบได้, มีการ training ให้กับผู้พัฒนา, มีการทำ document อย่างเต็มรูปแบบ, มี Help desk ไว้คอยช่วยเหลือและมีการส่ง team เพื่อช่วย support เช่น maintainer นั้นจะต้องทำการแสดงฟังก์ชันทั้งหมดของการ maintenance software และทำการแก้ปัญหาที่เกิดขึ้นทั้งหมด รวมทั้งปรับปรุงระบบตามการร้องขอของ user

Level 2 Corrective maintenance: จะคล้ายกับ Full maintenance แต่ว่าจะไม่มีการปรับปรุงเปลี่ยนแปลงระบบ จะมีเพียงการทำ corrective maintenance เท่านั้นจะไม่มีการทำการปรับปรุงเปลี่ยนแปลงระบบตามคำขอที่ต้องการจาก user อีก

Level 3 Limited corrective maintenance: จะคล้ายกับ Corrective maintenance แต่ว่าจะไม่มีการแก้ปัญหาที่มีความสำคัญสูงๆ เท่านั้น

Level 4 Limited software configuration management: จะทำการจำกัด function ที่จะทำการเปลี่ยนแปลงหรือแก้ไขกับระบบ เหมาะกับองค์กรที่มีงบประมาณน้อยมาก และถ้าเกิดงบประมาณมากขึ้น ก็สามารถเพิ่มระดับการ maintenance เพื่อเพิ่มประสิทธิภาพได้

2.10.3. รูปแบบของการ Maintenance

2.10.3.1 Corrective Maintenance คือการตรวจสอบความถูกต้องของระบบ และทำการแก้ไขข้อผิดพลาดที่เกิดขึ้นกับตัวระบบในการทำกิจกรรมต่างๆ บนระบบ

2.10.3.2 Adaptive Maintenance จะมีการทำการปรับปรุงเปลี่ยนแปลงค่าสภาพแวดล้อมที่มีอยู่ในระบบ

2.10.3.3 Perfective Maintenance จะทำการปรับปรุงระบบให้เกิดประสิทธิภาพสูงขึ้น โดยอาจจะเพิ่มคุณสมบัติต่างๆเพื่อให้ระบบมีประสิทธิภาพมากขึ้นได้

2.10.4 ปัจจัยของการ Maintenance ที่จะส่งผลกระทบ

Development process ก็คือ การที่รวมกระบวนการในการพัฒนา software และการที่จะทำการกำหนดและวางความสามารถของการ maintenance ไว้ ณ กระบวนการไหนที่จะให้เกิดประสิทธิภาพสูงสุด

Documentation คือการจัดการทำเอกสารต่างๆ จะช่วยลดต้นทุนเมื่อมีการเปลี่ยนแปลงหรือการปรับปรุงเกิดขึ้นกับระบบ รวมทั้งไม่ให้นักพัฒนารุ่นต่อไปต้องเสียเวลาในการที่จะรับช่วงต่อหรือทำการแก้ไขระบบ

Program Comprehensive คือความเข้าใจในตัวระบบ ถ้าเกิดผู้ที่จะรับผิดชอบนั้นไม่มีความเข้าใจในระบบ อาจเกิดจากไม่มีความรู้ความเข้าใจในระบบ, มีข้อมูลเกี่ยวกับระบบไม่

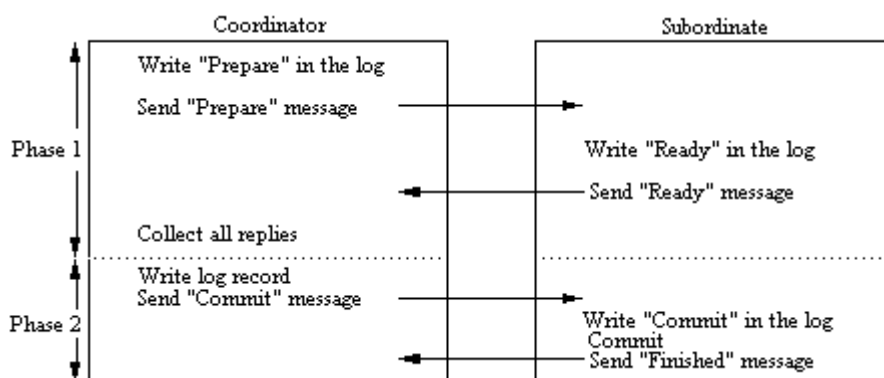
เพียงพอ เพราะฉะนั้นจะต้องมีการเลือกบุคคลที่จะมา maintenance ให้เหมาะสม รวมทั้งมีข้อมูลที่เพียงพอกับระบบ

2.11. โปรโตคอล Secure Socket Layer

SSL หรือ Secure Socket Layer คือ โปรโตคอล (ภาษาที่มีการกำหนดรูปแบบไว้อย่างเป็นกฎเกณฑ์เพื่อการติดต่อสื่อสารของระบบเครือข่าย) ที่ใช้บนเว็บสำหรับสร้างความปลอดภัยในการรับส่งข้อมูลระหว่างเว็บเบราว์เซอร์ และ เว็บเซิร์ฟเวอร์ โดยมีกลไก คือ การตรวจสอบซึ่งกันและกันระหว่างเว็บเบราว์เซอร์ และเว็บเซิร์ฟเวอร์ว่าผู้ใช้ทั้งสองฝ่ายเป็นผู้มีสิทธิ์จริง (Authorized parties) แล้วจึงทำการเข้ารหัสข้อมูล (Data Encryption) ที่รับส่งระหว่าง เบราว์เซอร์และเซิร์ฟเวอร์ เพื่อให้ข้อมูลเป็นความลับไม่สามารถถูกดักกลอบอ่านได้โดยง่ายจากผู้ที่ไม่มียสิทธิ์ (ในกระบวนการนี้ใบรับรองดิจิทัลจะถูกใช้ในการตรวจสอบ) โดยข้อมูลที่ถูกส่งนั้นจะถูกเข้ารหัสอยู่ในชั้นเครือข่าย (Network) ซึ่งอยู่ระหว่างชั้นโปรแกรมประยุกต์ (Application) กับชั้นทรานสปอร์ต ในโปรโตคอล TCP/IP ตัวอย่างที่นำ SSL มาใช้ ได้แก่ การชำระเงินด้วยบัตรเครดิตโดยการส่งข้อมูลบัตรเครดิตผ่านทางอินเทอร์เน็ต

2.12. ทุเฟสคอมมิต โปรโตคอล Two-Phase Commit Protocol

เป็นโปรโตคอลที่ใช้ดูแลการ commit ให้ทุกๆ site ทำการ commit พร้อมกัน โดยแต่ละ site จะต้องมีการ Transaction Manager ที่จะคอยดูแลเรื่องการ commit



รูปที่ 2.39. แสดงการทำงานของทุเฟสคอมมิต

ขั้นตอนการทำงาน ดังรูปที่ 2.39. มีขั้นตอนทำงานดังนี้คือ

เฟส 1

- Coordinator ทำการเขียนข้อความ "prepare to commit" ลงใน log file เพื่อเริ่มโปรโตคอล
- ทำการส่งข้อความให้กับแต่ละ participant ที่เกี่ยวข้อง
- ถ้า participant พร้อมที่จะ commit มันจะทำการเขียนข้อความ "ready" ลง log file แล้วส่ง ข้อความ "ready" ไปยัง Coordinator โดยถ้าตัดสินใจจะ commit หรือ abort แล้วจะเปลี่ยนกลับไม่ได้
- Coordinator จะรับข้อความจากแต่ละ participant เพื่อตัดสินใจว่าจะทำ Global Commit หรือ Global Abort

เฟส 2

- โดย Coordinator จะตัดสินใจเป็น Global Abort หากมี participant ใดๆ ตอบกลับเป็น Abort และจะตัดสินใจเป็น Global Commit เมื่อทุกๆ participant ตอบกลับเป็น Commit
- เมื่อตัดสินใจได้แล้วจะส่งข้อความออกไปยังแต่ละ participant
- เมื่อ Coordinator รับ ACK แล้วจึงจะค่อยลงมือเอา record ลง log file ว่า “commit complete” เพื่อป้องกันเรื่อง failure (Failure Detection)
- กรณีที่ participant เกิดล้ม เมื่อกลับเข้าสู่สภาพพร้อมทำงานแล้วจะต้องส่งข้อความไปถาม Coordinator อีกทีว่าเป็น Global Commit หรือ Global Abort